Hypo-Stage: An Empirical Study of Tool-Supported Architectural Hypothesis Engineering in Industrial Software Teams

José Gonçalves Lima Neto

Thesis presented to the
Institute of Mathematics,
Statistics, and Computer Science
of the University of São Paulo
in partial fulfillment
of the requirements
for the degree of
Master of Science

Program: Computer Science
Advisor: Prof. Dr. Paulo Roberto Miranda Meirelles

São Paulo
May, 2026

Hypo-Stage: An Empirical Study of Tool-Supported Architectural Hypothesis Engineering in Industrial Software Teams

José Gonçalves Lima Neto

This is the original version of the thesis prepared by candidate José Gonçalves Lima Neto, as submitted to the Examining Committee.

Acknowledgments

Abstract

José Gonçalves Lima Neto. **Hypo-Stage: An Empirical Study of Tool-Supported Architectural Hypothesis Engineering in Industrial Software Teams**. Thesis (Master's). Institute of Mathematics, Statistics, and Computer Science, University of São Paulo, São Paulo, 2026.

Software engineering teams frequently face architectural uncertainty, but lack lightweight, workflow-embedded support to record, prioritize, and learn from it collaboratively. The ArchHypo technique applies hypotheses engineering to architectural decisions; its manual application imposes a hard learning curve and depends strongly on expert facilitation. This present work reports the *first empirical account of a tool-mediated application of ArchHypo in an industrial setting*. The artifact is **Hypo-Stage**, an open-source Backstage plugin (https://github.com/ArchHypo/hypo-stage) deployed on a large organization's Internal Developer Portal, where engineers already work. The study adopts a flexible, multiple case-study design with formative evaluation and action-research elements, framed by Robson and McCartan's *Real World Research* perspective on flexible designs. Two teams—a Product team (twelve engineers) and a Platform team (four engineers)—used Hypo-Stage over two agile sprints (four weeks). Evidence was gathered through *participant observation*, *artifact analysis* of Hypo-Stage records, and open-ended *active listening sessions*. The work characterizes the types of architectural uncertainties teams made explicit, patterns of tool use and non-use, and perceived benefits and challenges across roles. From thematic analysis of the thirteen registered hypotheses, two families of patterns emerge: team-behaviour patterns describing the organizational conditions under which hypothesis engineering becomes sustainable (Group A), and five novel technical architectural hypothesis archetypes describing the recurring structural shapes that hypotheses take in practice (Group B) — the first empirically grounded contribution of this kind in the ArchHypo research line. *Refinements* to Hypo-Stage and its usage guidelines are treated as an integral part of the empirical contribution: each change is traced to concrete observations and linked to the research questions — especially RQ4 on empirically grounded improvement of the tool and its adoption.

Keywords: Software Architecture. Architectural Uncertainty. Hypotheses Engineering. ArchHypo. Hypo-Stage. Internal Developer Portal. Backstage. Case Study. Empirical Evaluation.

Resumo

José Gonçalves Lima Neto. **Hypo-Stage: Um Estudo Empírico sobre Engenharia de Hipóteses Arquiteturais Apoiada por Ferramenta em Equipes de Software Industriais**. Dissertação (Mestrado). Instituto de Matemática, Estatística e Ciência da Computação, Universidade de São Paulo, São Paulo, 2026.

Equipes de engenharia de software enfrentam com frequência incertezas arquiteturais, mas dispõem de pouco apoio fácil e bem integrado ao fluxo de trabalho para registrá-las, priorizá-las e aprender com elas de forma colaborativa. A técnica ArchHypo emprega engenharia de hipóteses nas decisões arquiteturais; sua aplicação manual envolve uma acentuada curva de aprendizado e depende fortemente de facilitação especializada de engenheiros experientes. O presente trabalho apresenta o *primeiro relato empírico de uma aplicação mediada por ferramenta de ArchHypo em contexto industrial.* O artefato é o **Hypo-Stage**, *plugin* open source para Backstage (https://github.com/ArchHypo/hypo-stage), implantado no Portal do Desenvolvedor Interno de uma grande organização de software, onde os engenheiros já atuam com Backstage. O estudo adota um desenho flexível de múltiplos estudos de caso, com propósito avaliativo formativo e elementos de pesquisa-ação, alinhado à perspectiva de *Real World Research* de Robson e McCartan sobre pesquisas com designs flexíveis. Duas equipes—uma equipe de Produto (doze engenheiros) e uma equipe de Plataforma (quatro engenheiros) — utilizaram o Hypo-Stage ao longo de duas *sprints* ágeis (quatro semanas). Os dados foram obtidos por meio de *observação participante*, *análise de artefatos* registrados no Hypo-Stage e sessões abertas de *escuta ativa* com os participantes. O trabalho caracteriza os tipos de incertezas arquiteturais que as equipes tornaram explícitas, padrões de uso e não uso da ferramenta, além de benefícios e desafios percebidos pelos diferentes papéis. A partir da análise temática das treze hipóteses registradas, emergem duas famílias de padrões: padrões de comportamento de equipes que descrevem as condições organizacionais sob as quais a engenharia de hipóteses se torna sustentável (Grupo A), e cinco arquétipos técnicos de hipóteses arquiteturais que descrevem as formas estruturais recorrentes que as hipóteses assumem na prática (Grupo B) — a primeira contribuição desse tipo com base empírica na linha de pesquisa do ArchHypo. Os *refinamentos* do Hypo-Stage e de suas diretrizes de uso são apresentados como parte integrante do resultado empírico: cada alteração é vinculada a observações concretas e relacionada às questões de pesquisa — em particular à RQ4, que trata dos refinamentos emergentes do estudo e de como eles respondem aos desafios identificados.

Palavras-chave: Arquitetura de Software. Incerteza Arquitetural. Engenharia de Hipóteses. ArchHypo. Hypo-Stage. Portal do Desenvolvedor Interno. Backstage. Estudo de Caso. Avaliação Empírica.

List of Abbreviations

SA	Software Architecture
ArchHypo	Architecture Hypothesis (technique)
IDP	Internal Developer Portal
NFR	Non-Functional Requirements
QA	Quality Attribute
DSR	Design Science Research
HE	Hypotheses Engineering
IME	Institute of Mathematics and Statistics (Universidade de São Paulo)
USP	University of São Paulo
ADR	Architecture Decision Record

List of Figures

List of Tables

List of Programs

Hypo-Stage — open-source Backstage plugin that operationalizes the ArchHypo technique for hypothesis management; empirical artifact of this thesis. Repository: https://github.com/ArchHypo/hypo-stage.

Backstage — open-source framework for Internal Developer Portals (CNCF); organizational platform in which Hypo-Stage is deployed in this study. Repository: https://github.com/backstage/backstage.

Contents

Chapter 1

Introduction

Software architecture (SA) plays a central role in the development and evolution of software-intensive systems. As firstly conceived by Perry and Wolf, 1992, the SA concept structures the system into elements and relations that form the basis of the system's organization as it must convey cohenrence, scalability, and maintainability for the relaying software over time. Despite the ever-changing practices in the modern era of software development, SA still provides the basis for analyzing the multiple quality attributes of nowadays software systems as it guides technical decisions that must be made throughout the development process over time via a diverse set of archictetual methodologies as the ones systematically mapped by Souza *et al.*, 2019. In organizations that rely on software as a core part of their business, architecture is not just a simple one-time design artifact, but a continuous decision-making process that must accommodate changing requirements, evolving technologies, and critical organizational constraints Silva, Melegati, Silveira, *et al.*, 2025.

In recent years, agile and continuous development practices have intensified this fast-paced pressure. Teams are expected to deliver more features in shorter cycles while preserving scalability, performance, and other quality attributes of their systems that continue to grow in size and complexity Silva, Melegati, Silveira, *et al.*, 2025; Silva, Melegati, Wang, *et al.*, 2024. In such environments, architecture decisions are frequently made under uncertainty: solutions or requirements may be incomplete or unstable, the impact of technology choices may not be fully understood, and the behavior of the system under realistic workloads may only become visible after deployment Silva, Melegati, Silveira, *et al.*, 2025. Despite this, architectural uncertainties often remain implicit and unmanaged, and decisions tend to rely on the tacit knowledge and intuition of a small group of experienced architects as oberseved by Souza *et al.*, 2019.

At the same time, many organizations seek to organize themselves around autonomous, cross-functional teams responsible for end-to-end delivery of features. Recent industry reports, such as the 2024 InfoQ Software Architecture and Design Trends Report Betts *et al.*, 2024, highlight socio-technical architecture as an emerging trend that emphasizes the co-design of technical and organizational structures, and the importance of considering how team boundaries and communication patterns shape system architectures. In this view,

architecture work is distributed across teams rather than centralized in a separate function or role. Moreover, in these scenarios, the Conway's Law is treated as a design constraint to be intentionally leveraged rather than an accident to be ignored Betts *et al.*, 2024. However, when architecture decisions depend on implicit assumptions held by a few experts in the organization, this distribution becomes difficult in practice. Less experienced team members may struggle to participate meaningfully in architectural discussions, and teams may find it hard to reason about the risks and trade-offs of alternative solutions even when they are applying anyone of thewell-known and validated architectural methodologies mapped by Souza *et al.*, 2019. This challenge reveals the case of the architectural uncertainty management, which still is a critical aspect of the architecture work, but is often neglected or poorly supported in practice by the available tools and methodologies.

1.1 Architectural Uncertainty and Hypotheses Engineering

Uncertainty in architecture decisions stems from limited knowledge about requirements, future usage scenarios, and the properties of candidate solutions as reviewed in Silva, Melegati, Silveira, *et al.*, 2025. Traditional risk management techniques assume that relevant events and their probabilities are known or can be estimated, which is not always the case when dealing with evolving systems and emerging technologies Silva, Melegati, Silveira, *et al.*, 2025. A complementary perspective is to acknowledge uncertainty explicitly and as an inevitable aspect of the architecture work, and therefore, to manage it as a first-class concern during the architecture work as proposed in Silva, Melegati, Silveira, *et al.*, 2025.

In this regard, Hypotheses Engineering has emerged as an approach to manage uncertainty in software development by treating assumptions as explicit, falsifiable hypotheses that can be prioritized and tested through experiments Silva, Melegati, Silveira, *et al.*, 2025; Silva, Melegati, Wang, *et al.*, 2024. In this context, ArchHypo is the technique that adapts hypotheses engineering to software architecture. It proposes that teams identify *architectural hypotheses* — statements capturing uncertainties that may affect architectural decisions — assess each hypothesis by its uncertainty level and impact, and define a technical plan for handling it Silva, Melegati, Silveira, *et al.*, 2025.

The technical plan may include actions to reduce uncertainty, such as experiments, architectural spikes, or tracer bullets, and actions to reduce the impact of potential changes, such as modularization or flexible deployment strategies Silva, Melegati, Silveira, *et al.*, 2025. By iteratively updating hypotheses and technical plans over time, ArchHypo aims to support a more deliberate and distributed approach to architectural decision-making, allowing teams to postpone irreversible decisions when justified and to focus their efforts on the most critical uncertainties Silva, Melegati, Silveira, *et al.*, 2025; Silva, Melegati, Wang, *et al.*, 2024.

Empirical studies have started to explore how ArchHypo works in practice. An experience report in a software start-up describes how the technique helped the founder identify and prioritize architectural uncertainties related to availability, usability, and scalability over a ten-month period, guiding the evolution of the product and avoiding

premature investments in infrastructure Silva, Melegati, Wang, *et al.*, 2024. A design science study in a mission-critical public-sector system shows that integrating ArchHypo into the development process enabled the team to distribute architectural work across iterations and to make better-informed decisions in the face of changing requirements Silva, Melegati, Silveira, *et al.*, 2025. At the same time, both studies report that the technique introduces a non-trivial learning curve and requires adjustments to existing processes, which might pose a challenge to its adoption by the team as reported in Silva, Melegati, Silveira, *et al.*, 2025; Silva, Melegati, Wang, *et al.*, 2024.

1.2 From Requirements to Architecture and the Role of Tool Support

The challenges of managing architectural uncertainty are closely related to the long-standing problem of deriving architectural models from requirements specifications. A systematic mapping study by Souza *et al.*, 2019 reviewed 39 methods for this derivation and concluded that existing approaches still rely heavily on the tacit knowledge of architects, provide limited support for less experienced practitioners, and often lack explicit evaluation mechanisms. Many methods describe high-level processes or frameworks, but offer little guidance on how to operationalize them in everyday practice as reported in Souza *et al.*, 2019.

The same study highlighted gaps in tool support. A significant portion of the methods did not include any supporting tool, and when tools were present, they tended to address only parts of the process as also reported in Souza *et al.*, 2019. The authors called for more automation, better mechanisms to capture and reuse architectural knowledge, and empirical validation of tools in realistic settings. These findings resonate with the observations from ArchHypo studies: while the technique provides a structured way to make architectural uncertainties explicit and to plan actions, applying it in practice is highly demanding without adequate supportting tool.

Grounded in these studies' insights, Corrêa, 2025 developed the Hypo-Stage, a support tool designed to help teams apply the ArchHypo technique in practice. Up to a specific point in its history, the work focused on designing and implementing the core functionality for enabling ArchHypo capabilities, such as representing architectural hypotheses, recording their uncertainty and impact assessments, and managing associated technical plans.[1] The work also discussed initial design decisions and limitations, including usability aspects and open questions about how the tool would fit into different team workflows. A more detailed description of Hypo-Stage and the contributions by Corrêa, 2025 are presented in Chapter 2.

[1] The first version of Hypo-Stage and the corresponding source code history up to commit `ceee509776508081ebdd473c2c4f710b8ef55947` document the initial design and implementation of Hypo-Stage proposed by Corrêa, 2025.

1.3 Problem Statement

When reviewed in intersection, these studies outline a clear problem. On the one hand, modern software organizations need to manage architectural uncertainty in a way that supports continuous evolution and involves the broader development team Silva, Melegati, Silveira, *et al.*, 2025; Silva, Melegati, Wang, *et al.*, 2024; Betts *et al.*, 2024. On the other hand, empirical evidence and systematic mapping studies indicate that:

- Methods for deriving and evolving architectures from requirements still depend strongly on architects' tacit knowledge and lack comprehensive tool support as reported in Souza *et al.*, 2019.
- ArchHypo offers a promising structure for handling architectural uncertainty, but teams perceive the technique as hard to learn and requiring process changes, especially without specialized support as reported in Silva, Melegati, Silveira, *et al.*, 2025; Silva, Melegati, Wang, *et al.*, 2024.
- Hypo-Stage provides an initial implementation of tool support for ArchHypo, but its impact on real software engineering teams and the refinements needed for effective use in industry remain unknown as reported in Corrêa, 2025.

In this context, there is a need to understand how software engineering teams, with diversed roles, tenures and responsibilities, interact with a tool like Hypo-Stage when working on their products and services. By forging this path, we seek to gain an understanding of what sorts of architectural uncertainties they manage through, and which aspects of the Hypo-Stage tool and its integration into the development process still require refinement. Addressing these concerns can contribute both to the practical adoption of ArchHypo via Hypo-Stage tool and to the broader discussion on how to support teams in sharing architectural responsibilities for accelerating adoption of the socio-technical architecture paradigm mapped in Betts *et al.*, 2024.

1.4 Objectives

The overall objective of this present work is to empirically evaluate and refine the Hypo-Stage tool, a support tool for the ArchHypo technique from Corrêa, 2025, by applying it with diverse software engineering teams in a real-world organization that maintains a scaled software product in Latin America. More specifically, this work aims to:

- Investigate how software engineering teams appropriate and use Hypo-Stage and ArchHypo in their day-to-day activities when working on an existing, scaled product.
- Characterize the architectural uncertainties that teams make explicit and manage through Hypo-Stage, and how these uncertainties evolve over time.
- Identify benefits and challenges perceived by team members regarding the use of Hypo-Stage, including its learning curve, usability, and fit with existing practices.
- Derive and implement refinements to Hypo-Stage and its usage guidelines based on empirical findings, with the goal of improving its suitability for supporting

architectural work in teams.

These objectives deliberately build on, rather than replicate, the previous studies' contributions in the ArchHypo research line. While the Corrêa, 2025 focused on designing and implementing Hypo-Stage, this work treats the tool as an existing artifact and concentrates on its evaluation and evolution in an industrial context.

1.5 Research Questions

To achieve the above objectives, this work is guided by the following research questions, listed in the same order in which they are organized in Chapter 5: tool use and appropriation (RQ1); recorded uncertainties and structural patterns in the hypothesis data (RQ2); perceived benefits and challenges (RQ3); and empirically grounded refinements to Hypo-Stage and its usage guidelines (RQ4).

RQ1: How do software engineering teams use Hypo-Stage and the ArchHypo technique when working on a scaled software product in a real-world organization?

RQ2: What types and recurring structural shapes of architectural uncertainties do these teams identify, record, and manage using Hypo-Stage, and what patterns emerge from their content and organizational context?

RQ3: What benefits and challenges do team members perceive in using Hypo-Stage to support architectural decision-making and uncertainty management?

RQ4: Which refinements to Hypo-Stage and its usage guidelines emerge from the empirical study, and how do they address the identified challenges?

Chapter 4 operationalises these questions through aligned data sources: participant observation and Hypo-Stage artefacts support RQ1 (tool use) and RQ2 (recorded uncertainties); active listening foregrounds RQ3 (perceived benefits and challenges) while also surfacing needs that motivate refinements under RQ4.

1.6 Research Contributions

By answering its set of research questions, this work attempts to make the following contributions:

- An empirical account of how multiple software engineering teams in a real organization use a tool-supported implementation of ArchHypo (via Hypo-Stage tool) while working on a scaled software product, including observed practices, adaptations, and obstacles as reported in Corrêa, 2025.
- A characterization of architectural uncertainties managed by these teams, describing their sources, quality attributes, and evolution, as captured through Hypo-Stage as reported in Corrêa, 2025.
- A set of five novel *technical architectural hypothesis archetypes* (Group B patterns) derived from thematic analysis of the thirteen hypotheses registered during the study.

RQ	Research Question	Primary Objective Addressed
RQ1	How do software engineering teams use Hypo-Stage and ArchHypo when working on a scaled product?	Investigate appropriation and use patterns of Hypo-Stage in day-to-day team activities.
RQ2	What types and recurring structural shapes of architectural uncertainties do teams identify, record, and manage using Hypo-Stage, and what patterns emerge from their content and organizational context?	Characterise architectural uncertainties by source, quality attribute, and lifecycle; identify recurring structural archetypes of hypotheses that emerge from the empirical data as a novel contribution to the ArchHypo pattern literature.
RQ3	What benefits and challenges do team members perceive in using Hypo-Stage?	Identify perceived benefits and challenges related to learning curve, usability, and workflow fit.
RQ4	Which refinements emerge from the empirical study and how do they address identified challenges?	Derive and implement empirically grounded refinements to Hypo-Stage and its usage guidelines.

Table 1.1: *Alignment between research questions and study objectives.*

These archetypes — Hypothesis Cluster, Regulatory Trigger Hypothesis, Decoupling Seam Hypothesis, Load-Validated Hypothesis, and Shared Resource Governance Hypothesis — constitute the first empirically grounded answer to the question of what recurring shapes architectural hypotheses take when ArchHypo is applied in practice, addressing a gap explicitly identified as future work in the prior ArchHypo pattern literature (Silva *et al.*, 2024).

- A set of empirically grounded refinements to the Hypo-Stage tool and its usage guidelines, aimed at reducing the learning curve, improving usability, and better integrating the technique into team workflows as reported in Corrêa, 2025.
- Reflections on the implications of these findings for supporting distributed architectural work in agile and continuous development settings, connecting the ArchHypo experience to broader concerns about tacit knowledge and tool support in software architecture as reported in Souza *et al.*, 2019; Betts *et al.*, 2024.

1.7 Research Structure

The remainder of this work is organized as follows.

Chapter 2 provides the theoretical foundations for the study. It introduces key concepts in software architecture and architectural uncertainty; presents the ArchHypo technique in detail, including its core concepts, assessment model, technical plan strategies, and lifecycle; reviews the two prior empirical evaluations of ArchHypo (at Catch Solve and PropSEP/Jetsoft); describes Internal Developer Portals and the Backstage platform as the

delivery vehicle for Hypo-Stage; presents the initial Hypo-Stage tool developed by Corrêa, 2025; and surveys the systematic mapping study by Souza *et al.*, 2019 on the gap between architectural derivation methods and tool support.

Chapter 3 synthesizes related literature and industry context relevant to tool-supported architectural decision-making and positions this work within that space.

Chapter 4 presents the research design and methods adopted in this dissertation. It describes the research strategy (a flexible multiple case study with formative evaluation and action research elements), the conceptual research framework that organizes the key constructs of the study, the industrial context and case selection rationale for the two participating teams (a Product Team and a Platform Team), the researcher's dual role as participant observer and facilitator, the data collection procedures (participant observation, artifact analysis, and open-ended active listening sessions), the thematic coding analysis approach, and the criteria used to establish trustworthiness and ethical conduct.

Chapter 5 presents the empirical results and refinements of the study. The chapter follows the research questions in order: how teams used Hypo-Stage and ArchHypo in practice (RQ1); the architectural uncertainties and hypotheses they recorded, including aggregate data characterization and Group B structural patterns (RQ2); the benefits and challenges they perceived (RQ3); and the refinements to Hypo-Stage and its usage guidelines that emerged from the study, each traced to observations and linked primarily to RQ4. Presenting results and refinements together makes explicit how empirical findings drove iterative tool evolution throughout the observation period.

Finally, Chapter 6 builds on the integrated empirical account in Chapter 5: it summarizes the main contributions of the work in response to each research question, discusses implications for practitioners adopting hypothesis-engineering tools and for researchers studying tool-mediated architectural practices, and outlines directions for future work on Hypo-Stage and on broader support for distributed architectural decision-making in agile organizations.

Chapter 2

Background

This chapter establishes the theoretical foundations for the empirical investigation carried out in this present work. It begins with an overview of software architecture and architectural uncertainty, then introduces hypotheses engineering and the ArchHypo framework in technical details, describes Internal Developer Portals and their role as platforms for enabling tools such as Hypo-Stage, presents the Hypo-Stage tool itself, and concludes with a summary of findings from Souza *et al.*, 2019 who mapped the gaps between architectural derivation methods and practical tool support.

2.1 Software Architecture and Architectural Uncertainty

Software architecture basically refers to the high-level structure of a software system, encompassing its components, their relationships, and the design principles that guide its evolution L. Bass *et al.*, 2021. However, in organizations that maintain software at the core of their business developed and operated by multiple teams, the ever-happening architectural decisions shape not only technical quality attributes such as performance, reliability, and maintainability of systems, but also the ability of engineering teams to work independently and deliver changes quickly and efficiently at a speed that needs to match the speed of the business Skelton and Pais, 2019.

More embracing to that rapidly changing business scenarios, Richards and Ford, 2020 added that the design decisions themselves are part of the software architecture itself, making it not only a static structure or frozen blueprint of a system, but a living body of reasoning that is constantly being refined and adapted to the changing business needs. This work sees that modern definition as a way of forging the role of Architectural Uncertainty: the rationale behind any architectural choice is valid only relative to the information available at the time of the decision, and that information is always incomplete and subject to change.

In other words, Architectural Uncertainty arises whenever teams must make structural decisions under incomplete information. This is a normal condition in continuous-delivery

environments where business requirements, user behavior, and technology options are only partially known. When uncertainty is left implicit, teams either defer decisions indefinitely (accumulating technical debt) or make premature actions that may constrain the architecture evolution of their systems. Both outcomes can slow down delivery and degrade system quality over time, turning the software architecture into a big ball of mud as perceived by Foote and Yoder, 1999 in the late 90s.

Moreover, the concept of "fast flow", popularized by the Team Topologies framework in Skelton and Pais, 2019, emphasizes that organizations should design both their teams and their architecture to enable a continuous, rapid stream of software changes. Achieving fast flow presupposes that teams can make local architectural decisions without waiting for a central architect to approve them. This in turn demands that architectural uncertainties be made explicit and that lightweight mechanisms exist for teams to assess, prioritize, and revisit them as evidence accumulates. This is where Hypotheses Engineering may be used to play a key role in the architecture work, as proposed by Silva, Melegati, Silveira, *et al.*, 2025.

2.2 Hypotheses Engineering and ArchHypo

Hypotheses engineering is an approach to managing technical uncertainty by formulating architectural decisions as testable hypotheses rather than settled conceived facts. Instead of asserting that a particular technology or pattern *will* satisfy a quality requirement, teams state that it *might* do so and then design experiments to gather evidence to validate or invalidate the hypothesis.

The ArchHypo technique, proposed in Silva, Melegati, Silveira, *et al.*, 2025, operationalized this idea for software architecture. Grounded in Design Science Research (DSR) and evolved from a body of patterns for agile architecture first disseminated at conferences and through corporate training reported in Silva, Melegati, Wang, *et al.*, 2024, ArchHypo provides a structured vocabulary and a set of practices that fit within iterative development workflows where architecture uncertainty is a constant challenge for rapidly-evolving software systems.

2.2.1 Core Concepts

An *architectural hypothesis* is a falsifiable statement that captures an uncertainty relevant to the design of a software architecture. It is important to distinguish the concept from adjacent concepts:

- **Hypothesis versus Risk:** A risk involves a known event with estimable probability and impact. An architectural hypothesis, by contrast, stems from a *lack of information* sufficient to make a decision. The distinguishing feature is the epistemic gap, not the probability of an outcome as observed in Silva, Melegati, Silveira, *et al.*, 2025.
- **Hypothesis versus Requirement:** A requirement specifies what the system must do or how well it must do it. A hypothesis may be triggered by a requirement (e.g., a quality-attribute goal that cannot yet be confirmed as achievable), but the

hypothesis captures the uncertainty about the *path* to satisfying that requirement, not the requirement itself.

- **Hypothesis versus Architectural Decision.** An architectural decision resolves a design choice. A hypothesis *precedes and informs* decisions: it identifies what must be learned or validated before a decision can responsibly be made (Silva, Melegati, Silveira, *et al.*, 2025).

Concept	Definition	Relation to Architectural Hypothesis
Risk	A known event with estimable probability and impact.	A hypothesis stems from an *epistemic gap* — lack of information — not from a known probability.
Requirement	Specifies what the system must do or how well it must do it.	A requirement may *trigger* a hypothesis when the path to satisfying it is uncertain; they are not the same artifact.
Architectural Decision	Resolves a design choice.	A hypothesis *precedes and informs* a decision; it captures what must be learned before a decision can responsibly be made.

Table 2.1: *Distinguishing architectural hypotheses from adjacent concepts (Silva, Melegati, Silveira, et al., 2025).*

Hypotheses arise from two principal sources:

- (a) *requirements*: when quality attribute targets are poorly specified or the team lacks information about whether a given approach can satisfy them; and
- (b) *solutions*: when the consequences of an existing or candidate architectural solution are unknown.

2.2.2 Assessment

Each hypothesis is evaluated along two independent dimensions:

1. **Uncertainty level.** How far the team is from being able to refute or accept the hypothesis. Crucially, uncertainty level is *not* equivalent to probability. A hypothesis that is almost certainly true and a hypothesis that is almost certainly false both represent *low* uncertainty, because in either case the team already has enough information to act. High uncertainty means the team genuinely does not know which way the evidence will fall.
2. **Impact level.** The potential consequences –— in terms of rework, schedule, cost, or quality degradation –— if the hypothesis is invalidated. A high uncertainty combined

with a high impact signals that a hypothesis demands urgent attention.

Both dimensions can be assessed using a 3-point or 5-point qualitative scale; assessments are made independently and relatively, not on absolute probability as reported in Silva, Melegati, Silveira, *et al.*, 2025.

2.2.3 Technical Plans

For hypotheses with significant uncertainty or impact, teams formulate a *technical plan*: a set of concrete actions whose goal is to (a) accept or refute the hypothesis, (b) reduce its uncertainty level, (c) reduce its potential impact, or (d) define criteria under which the decision can responsibly be deferred. ArchHypo identifies the following plan strategies as observerd in Silva, Melegati, Wang, *et al.*, 2024:

- **Architectural Spike.** A time-boxed, isolated experiment to explore a technology or design option without coupling the result to the production codebase. The spike produces information rather than deliverable functionality.

- **Tracer Bullet.** A thin, end-to-end implementation within the actual system, exercising all architectural layers to validate an integration or performance characteristic under realistic conditions.

- **Software Analytics.** Using monitoring, profiling, or usage data already collected from the running system to gather evidence about a hypothesis without additional development effort.

- **Modularity / Flexibility.** Restructuring the affected subsystem to reduce coupling or increase replaceability, thereby lowering the *impact* of the hypothesis rather than resolving its uncertainty. This strategy is useful when full resolution is premature but protection against a bad outcome is warranted.

- **Guideline Implementation.** Tailoring the development process—for example, adopting a coding standard, a review protocol, or a testing policy—so that the team systematically generates evidence about the hypothesis as a by-product of normal work. This was the most frequently used strategy in the evaluation reported in Silva, Melegati, Silveira, *et al.*, 2025.

- **Architectural Trigger.** A condition-based deferral in which the team agrees to act on the hypothesis only when a specific observable event occurs (e.g., when the number of concurrent users exceeds a threshold, or when a performance regression appears in monitoring). Until the trigger fires, the hypothesis is deliberately deferred.

- **Agile Landing Zones.** Adopting architectural patterns that explicitly preserve flexibility, creating "landing zones" into which future decisions can be inserted without major disruption, regardless of which direction the evidence eventually points.

Strategy	Description	Primary Effect
Architectural Spike	Time-boxed isolated experiment; produces information, not deliverable functionality.	Reduces uncertainty.
Tracer Bullet	Thin end-to-end implementation in the real system to validate integration or performance.	Reduces uncertainty under realistic conditions.
Software Analytics	Uses existing monitoring or profiling data to gather evidence.	Reduces uncertainty without extra development effort.
Modularity / Flexibility	Restructures the subsystem to reduce coupling or increase replaceability.	Reduces impact (not uncertainty).
Guideline Implementation	Tailors the development process (coding standards, review protocols) to generate evidence as a by-product of normal work.	Reduces uncertainty incrementally; most frequently used (Silva, Melegati, Silveira, *et al.*, 2025).
Architectural Trigger	Defers action until a specific observable condition is met.	Manages timing of decision; reduces premature cost.
Agile Landing Zones	Adopts patterns that preserve flexibility for future insertions.	Reduces impact proactively.

Table 2.2: *ArchHypo technical plan strategies and their primary effect (Silva, Melegati, Wang, et al., 2024; Silva, Melegati, Silveira, et al., 2025).*

2.2.4 Lifecycle

Hypotheses progress through a defined lifecycle: they begin in an *open* state and transition to either *validated* or *invalidated* as evidence from technical plans accumulates. The responsible-moment is the principle—act of the team during the hypothesis lifecycle. It is when they have enough information, neither too early nor too late, to take the effort to move forward with the architectural decision or defer it further.

2.2.5 Empirical Evaluations

IEEE Software evaluation (Catch Solve startup). The first evaluation of ArchHypo in a real operating environment was reported in an IEEE Software article in Silva, Melegati, Wang, *et al.*, 2024. The setting was Catch Solve, a startup offering web testing and monitoring services based in Italy's South Tyrol region. Four architectural hypotheses were formulated and tracked over approximately ten months, covering quality attributes of availability, reusability of test templates, usability, and scalability of virtual machines.

For example, the hypothesis *"The lack of redundancy can cause problems with application availability for the customers"* was assessed with a low impact and very low uncertainty, which led the team to defer investment in redundant infrastructure and instead set up an Architectural Trigger based on the number of new customers. The study showed that naming and tracking uncertainties explicitly helped the technical lead decide when to invest in architectural improvements versus when to wait, and validated the overall structure of the ArchHypo approach. A key insight from this evaluation was that explicit hypothesis management changed how the team prioritized technical work: rather than treating architectural uncertainty as urgent and tacit, it became visible, assessed, and deliberately managed within a defined lifecycle.

TSE evaluation (PropSEP at Jetsoft). The larger and more controlled evaluation, published in the IEEE Transactions on Software Engineering Silva, Melegati, Silveira, *et al.*, 2025, examined the application of ArchHypo within Jetsoft, a Brazilian software development company, on the PropSEP project—the budget proposal system of the Planning and Management Secretariat of the State of São Paulo. PropSEP is mission-critical: the approved budget it processes amounts to approximately 317 billion BRL Silva, Melegati, Silveira, *et al.*, 2025. The evaluation ran across six sprints of seven business days each (May–August 2022) and involved eight participants (P1–P8), all holding computer science degrees with certifications in Oracle, WebSphere, or GeneXus. Ten architectural hypotheses were identified, spanning six quality attributes (performance, security, reliability, flexibility, usability, and team productivity), and seven distinct types of architectural practice were employed in the resulting technical plans Silva, Melegati, Silveira, *et al.*, 2025.

The study produced nine key findings:

1. Some hypotheses carried uncertainty that was monitored throughout the entire project duration without reaching full resolution, demonstrating that the technique accommodates long-horizon uncertainties Silva, Melegati, Silveira, *et al.*, 2025.
2. ArchHypo enabled the team to avoid upfront architectural design, distributing structural decisions across iterations as evidence became available Silva, Melegati, Silveira, *et al.*, 2025.
3. Participants reported improvements to the development process in terms of safety, transparency, and predictability Silva, Melegati, Silveira, *et al.*, 2025.
4. Hypotheses spanning different quality attributes were handled using qualitatively different technical strategies, illustrating the creativity the technique affords Silva, Melegati, Silveira, *et al.*, 2025.
5. Guideline Implementation (process tailoring) emerged as the most frequently employed strategy across all technical plans Silva, Melegati, Silveira, *et al.*, 2025.
6. The team collectively recognized that managing hypotheses improved the quality of their architectural decision-making Silva, Melegati, Silveira, *et al.*, 2025.
7. Post-release evidence pointed to a positive impact on product quality: no issues related to the managed hypotheses were detected after deployment Silva, Melegati, Silveira, *et al.*, 2025.

8. **Changes in the team's way of working constituted a significant adoption obstacle.** Mixing architectural and hypothesis-related tasks with functional development created estimation difficulties, and participants described the additional work as not feeling directly connected to product delivery Silva, Melegati, Silveira, *et al.*, 2025.

9. **All eight participants unanimously considered the technique hard to learn.** Specific difficulties included: mapping architectural concerns to the hypothesis format, formulating appropriate technical plan actions, articulating the assumptions behind a hypothesis, and understanding hypotheses as the primary driver of architectural action. No participant felt prepared to apply ArchHypo without continued expert support after the initial workshop Silva, Melegati, Silveira, *et al.*, 2025. One participant explicitly stated that *"more training would make the team less dependent on the team of architects"* (P3) Silva, Melegati, Silveira, *et al.*, 2025. The authors themselves acknowledge this structural dependency in their discussion. The absence of a supporting tool that encodes this knowledge into the workflow risks ArchHypo to remain a technique that only functions when an experienced architect is actively steering its application.

Findings 8 and 9 directly motivate the tooling contribution of this present work: a tool that embeds ArchHypo scaffolding in the daily engineering workflow, reducing the cognitive overhead of manual application and knowledge transfer.

Study	Setting	Scope	Key Finding	Limitation
Silva, Melegati, Wang, *et al.* (2024)	Catch Solve startup (Italy)	4 hypotheses, 10 months	Explicit hypothesis tracking changed how team prioritized architectural investments.	Single practitioner; no tool support.
Silva, Melegati, Silveira, *et al.* (2025)	PropSEP / Jetsoft (Brazil)	10 hypotheses, 6 sprints, 8 participants	ArchHypo distributed architectural work across iterations; all participants found technique hard to learn.	Manual application; expert steering required; no supporting tool.
This work	Large-scale tech organization (Latin America)	Multiple teams, scaled product	To be reported — first evaluation in a tool-mediated setting via Hypo-Stage.	Single organizational context; Backstage dependency.

Table 2.3: *Summary of prior empirical evaluations of ArchHypo.*

2.3 Internal Developer Portals and Backstage

The adoption of microservice architectures and the pursuit of team autonomy through practices such as Team Topologies have created significant complexity in the daily workflow of software engineering teams. In response, the industry has converged around the concept of *Internal Developer Portals* (IDPs) as a platform-engineering discipline aimed at reducing this complexity while preserving team independence Corrêa, 2025.

2.3.1 The Role of Internal Developer Portals (IDPs)

An Internal Developer Portal serves as the primary interface between developers and the underlying platform infrastructure. Unlike a simple wiki or a project-management tool, an IDP is an operational hub that aggregates tools, services, documentation, and data into a unified "pane of glass" Corrêa, 2025. Its primary goal is to reduce *cognitive load*: by abstracting the complexity of infrastructure and standardizing workflows, IDPs enable what practitioners call *Golden Paths* – recommended, supported, and automated paths for building and deploying software Corrêa, 2025.

This goal aligns directly with the theory of autonomous stream-aligned teams proposed by Skelton and Pais, 2019: if teams are to own full slices of a product without depending on a central gatekeeper, the platform must lower the barriers to acting on that ownership. An IDP operationalizes this by ensuring that the information and tooling a team needs—service catalog, documentation, CI/CD pipelines, monitoring dashboards, and domain-specific plugins—are accessible from a single environment, without context-switching.

Conceptually, IDPs sit at the intersection of two ideas that are highly relevant to this work. First, they embody the principle of *decentralized architecture*: rather than requiring teams to escalate architectural concerns to a central architecture group, a well-designed IDP makes it possible to encode architectural guidance, constraints, and tools directly into the developer workflow. Second, they are the natural delivery vehicle for hypothesis-engineering support: because ArchHypo must become part of the daily engineering cadence — not a separate off-sprint activity —embedding it in an IDP portal where developers already spend their working time is strategically motivating.

2.3.2 Backstage: Open Platform for Internal Developer Portals

Backstage[1] is an open-source framework for building Internal Developer Portals, originally developed at Spotify to manage the complexity arising from rapid growth and an expanding microservice estate. Spotify's internal version of Backstage serves as the central nervous system for its infrastructure, used by over 2,700 engineers to manage more than 14,000 software components Spotify and Cloud Native Computing Foundation, 2024. Spotify open-sourced the framework in March 2020, and it was subsequently donated to the Cloud Native Computing Foundation (CNCF) as an Incubating project Spotify and Cloud Native Computing Foundation, 2024. As of 2025, more than 3,000 companies have adopted Backstage for their own portals Spotify and Cloud Native Computing Foundation, 2024.

[1] https://github.com/backstage/backstage

Backstage is not a rigid application but rather a modular ecosystem driven by configuration. Its architecture rests on three layers: (1) the *Core Framework*, which manages essential utilities including the plugin loader, routing, authentication, and error handling; (2) the *Plugin Ecosystem*, built around a split-plugin model where frontend plugins handle views and UI components while backend plugins manage API services, data processing, and storage; and (3) *External Integrations*, through which backend plugins serve as gateways to external systems and APIs, allowing the portal to aggregate data from cloud providers, CI/CD tools, and other infrastructure Corrêa, 2025.

Layer	Responsibilities	Relevance to Hypo-Stage
Core Framework	Plugin loader, routing, authentication, error handling.	Provides the lifecycle hooks that Hypo-Stage's frontend and backend plugins register into.
Plugin Ecosystem	Frontend plugins (views, UI); backend plugins (API services, data, storage).	Hypo-Stage is implemented as a split plugin: the frontend renders hypotheses and assessments; the backend persists data and exposes the API.
External Integrations	Backend plugins as gateways to external systems (cloud, CI/CD, APIs).	Hypo-Stage integrates with the Backstage catalog to auto-associate hypotheses with software entities.

Table 2.4: *Backstage architecture layers and their role (Corrêa, 2025).*

Crucially, the plugin architecture allows organizations to extend the portal's functionality by integrating custom tools directly into the developer's daily workflow, without forcing context-switching. It is precisely this extensibility that makes Backstage the appropriate delivery vehicle for the Hypo-Stage tool: a plugin can introduce a new class of first-class object – the architectural hypothesis – into the same environment where a developer already inspects service health, reviews documentation, and triggers CI/CD runs Corrêa, 2025.

Corrêa, 2025 demonstrated the technical feasibility of building such a plugin and validated the integration approach. That work established the initial version of the Hypo-Stage plugin for Backstage, which constitutes the artifact under empirical evaluation in this present work.

2.4 Hypo-Stage: The Initial Tool

Hypo-Stage is an open-source Backstage plugin developed as an undergraduate capstone project by Pedro Henrique Mariano Corrêa, supervised by the author of this present work and co-supervised by Prof. Paulo Meirelles Corrêa, 2025. Its purpose is to opera-

tionalize ArchHypo within the Spotify's IDP Backstage, embedding hypothesis management into the engineering platform workflow.

The initial version of Hypo-Stage, corresponding to the code up to commit `ceee509776508081ebdd473c2c4f710b8ef55947` in the `hypo-stage` repository,[2] provides the following capabilities:

- **Hypothesis management**: creating, editing, listing, and deleting architectural hypotheses, each linked to a Backstage catalog entity (e.g., a service, component, or system). Integration with the Backstage catalog means that hypotheses are automatically associated with the software entities they affect, without requiring separate registration.
- **Uncertainty and impact assessment**: rating each hypothesis on 5-point Likert scales for both uncertainty level and impact level, with free-text rationale fields. The use of a 5-point scale rather than a binary or 3-point scale provides finer-grained prioritization while remaining tractable for teams without formal risk management backgrounds.
- **Technical planning**: attaching one or more technical plan items to a hypothesis, specifying action type (spike, tracer bullet, trigger, etc.), description, target date, expected outcome, and documentation links.
- **Visualization**: a chart plotting hypotheses on an uncertainty-vs-impact plane to support prioritization, making the relative urgency of hypotheses visible at a glance.
- **Status lifecycle**: tracking each hypothesis through the defined lifecycle states (open → validated or invalidated), preserving the history of status transitions to support retrospective analysis.

By integrating with Backstage, Hypo-Stage leverages the portal's service catalog so that hypotheses are automatically associated with the software entities they affect. This design reduces context-switching: developers manage architectural hypotheses in the same environment where they consult documentation, inspect service health, and access CI/CD pipelines.

Known limitations: The initial tool design assumed familiarity with the ArchHypo vocabulary, which poses a barrier for teams new to hypothesis engineering. The case study by Corrêa, 2025 reconstructed the Catch Solve startup scenario digitally but did not involve real-time use by a team. Furthermore, the tool lacked features for collaborative assessment (e.g., multiple raters per hypothesis) and for tracking the evolution of assessments over time. These gaps also motivated the empirical evaluation and refinement effort promoted in this present work.

[2] Source repository: https://github.com/ArchHypo/hypo-stage. The baseline revision for the initial tool design referenced throughout this thesis is commit `ceee509776508081ebdd473c2c4f710b8ef55947`.

Capability	Description	Known Limitation
Hypothesis management	Create, edit, list, delete hypotheses linked to Backstage catalog entities.	Assumed prior ArchHypo vocabulary familiarity; no onboarding guidance.
Uncertainty & impact assessment	5-point Likert scales per dimension with free-text rationale.	No collaborative multi-rater assessment; evolution of assessments not tracked over time.
Technical planning	Attach plan items with action type, description, target date, and outcome.	Action types not enforced or explained in-tool; required expert facilitation to use correctly.
Visualization	Uncertainty-vs-impact scatter plot for prioritization.	Static; not updated reactively as assessments changed.
Status lifecycle	Tracks hypotheses through open → validated / invalidated states.	No history of intermediate state transitions preserved.

Table 2.5: *Hypo-Stage initial capabilities and known limitations identified prior to this study (Corrêa, 2025).*

2.5 Deriving Architecture from Requirements: The Tooling Gap

The systematic mapping study by Souza *et al.*, 2019 examined the existing body of work on methods for deriving architectural models from requirements specifications. Starting from 2,575 candidate papers, the authors applied a PICOC (Population, Intervention, Comparison, Outcome, Context) search structure and a quality assessment checklist – composed of general criteria (weighted at 25%) and method-specific criteria (weighted at 75%) – to select 39 high-quality primary studies for detailed analysis. Their analysis employed the NIMSAD (Normative Information Model-based Systems Analysis and Design) framework to characterize the structure of each reviewed method.

Among their key findings, these were directly relevant to this work:

1. **Dependence on tacit knowledge.** Every reviewed method relied, to some degree, on the architect's experience and intuition. No method provided a fully automated or guided derivation path accessible to less-experienced practitioners Souza *et al.*, 2019.

2. **Limited support for less-experienced practitioners.** Methods assumed a skilled architect capable of navigating trade-offs among quality attributes. Teams without senior architectural expertise had little actionable guidance Souza *et al.*, 2019.

3. **Weak or absent tool support.** Although some tools were identified—most notably QuaDAI and Monarch—they addressed only fragments of the derivation process and offered only partial coverage of the architectural reasoning cycle. The mapping found no comprehensive tool integrating hypothesis formulation, assessment, and tracking within a team's daily workflow Souza *et al.*, 2019.
4. **Insufficient evaluation.** Many proposed methods lacked rigorous empirical validation, relying on toy examples or author-run case studies rather than independent evaluations with real teams in production environments Souza *et al.*, 2019.

Souza *et al.*, 2019 also noted that 76.9% of the 39 primary studies addressed functional requirements, with a significant proportion also treating non-functional requirements; yet the methods for handling quality attributes — precisely the domain where architectural uncertainty is most consequential — were among the least well supported by tools or empirical evidence.

These findings reinforce the motivation for this work on two fronts. First, they confirm the existence of a "tooling void" in which architectural management remains disconnected from the daily engineering workflow. Second, they underscore the need for empirical evaluations with real teams – precisely the kind of evaluation this work aims to provide for the Hypo-Stage tool and ArchHypo.

Chapter 3

Related Work

This chapter synthesizes the literature relevant to the empirical evaluation of the Hypo-Stage tool. Rather than presenting a standalone systematic review, it draws on the author's earlier exploratory work — which included a tertiary systematic mapping study on software architecture for startups and a multivocal literature review on centralization and decentralization of architectural decision-making — and repositions those findings as narrative context for this work. The chapter also incorporates references from the grey literature. Such sources are explicitly identified as relevant, despite having weaker evidential weight than peer-reviewed publications, because they often capture industry trends and real organizational experience more rapidly than academic publication cycles allow.

3.1 Centralization and Decentralization of Architectural Decisions

A recurring theme in both academic and gray literature is the tension between centralizing architectural authority in a few experienced individuals versus distributing it across development teams. Organizations pursuing fast flow and team autonomy increasingly favor decentralized models as observed by Salameh and J. M. Bass, 2021; Oliveira De Diana *et al.*, 2011, yet several studies report that without explicit governance mechanisms, decentralization leads to inconsistent decisions and architectural drift as seen in the work of Da Silveira Neto *et al.*, 2025; Dasanayake *et al.*, 2017; Oliveira De Diana *et al.*, 2011.

The author's preliminar exploratory work explored this tension through two complementary lenses. Firstly, the author conducted a tertiary systematic mapping study surveying the existing landscape of secondary studies on software architecture for startup and scale-up organizations, identifying recurring themes in how early-stage teams approach architecture governance under resource and time constraints. For the second lens, the author performed a multivocal literature review that synthesized both academic publications and grey literature (practitioner blogs, conference talks, and industry reports), covering three interrelated perspectives: deriving software architecture from requirements, the emerging socio-technical architecture trend (which situates architectural decisions within organizational and team structures), and the tension between centralized and

decentralized decision-making.

The multivocal review surfaced a framing where the industry advocates strongly for decentralized decisions to enable team autonomy and fast flow Skelton and Pais, 2019, yet the methods and tools available for architectural decision-making depend heavily on the tacit knowledge of experienced architects as observed by Souza *et al.*, 2019. Decentralization thus risks transferring decision responsibility to practitioners who lack the conceptual vocabulary or methodological support to exercise it effectively. However, the author recognized that this tension reflects a recognized design trade-off rather than a genuine logical contradiction, and building this work around resolving this tension presented a risk of over-extending the scope of this work's contribution. For this reason, this present work consequently does not employ that framing; instead, it focuses on the concrete, tractable problem of providing tool support for one well-defined uncertainty management technique.

3.2 Socio-Technical Architecture

A significant body of work, both academic and practitioner-oriented, frames architectural decision-making not merely as a technical concern but as a *socio-technical* one — shaped by the organizational structure, communication patterns, and cognitive constraints of the teams involved, as observed in Tang *et al.*, 2017; Razavian, 2019; Herbsleb and Grinter, 1999. This framing is directly relevant to the motivation for the Hypo-Stage tool, because tool-embedded scaffolding does not only lower the technical cost of applying the ArchHypo technique, but it also contributes to changing the social dynamics of how architectural uncertainty is surfaced and addressed within a team Borowa *et al.*, 2023; Razavian, 2019.

3.2.1 Conway's Law and Its Implications

The foundational observation underlying the socio-technical framing is Conway's Law Conway, 1968: "Any organization that designs a system (defined broadly) will produce a design whose structure is a copy of the organization's communication structure." Conway's Law implies that architectural decisions cannot be analyzed in isolation from the organizational and team structure in which they are made. Considering that perspective, this work also supports the idea that team boundaries become architectural boundaries.

Skelton and Pais, 2019 extended this insight into an actionable framework with the concept of the *Inverse Conway Maneuver*: rather than allowing the organization's structure to passively determine the architecture, teams should deliberately design team topologies to produce the target architecture. In practical terms, this means that architectural governance — including how uncertainty is managed and who is empowered to make decisions — must be considered a design decision with organizational as well as technical consequences.

3.2.2 Socio-Technical Architecture as an Industry Trend

The InfoQ Software Architecture and Design Trends Report for 2024 by Betts *et al.*, 2024 identified *socio-technical architecture* as a distinct "Early Adopter" trend, explicitly

describing it as replacing the earlier framing of "architecture as a team sport." The report notes that the trend encompasses two related ideas: on one side, the evolving role of architects as mentors and technical leaders rather than sole decision-makers; on the other, the growing recognition — driven in part by the dissemination of Team Topologies Skelton and Pais, 2019 — that the design of a system must consider all the people who build, support, and maintain it Betts *et al.*, 2024.

It is important to note that InfoQ is a practitioner-oriented publication, not a peer-reviewed academic venue. The Trends Report reflects the editorial judgment of an experienced editorial board drawing on community submissions, conference talks, and practitioner case studies. As grey literature, its findings carry weaker evidential weight than systematic empirical studies; nonetheless, they are valuable as an indicator of what concepts are gaining traction in industrial practice and what challenges practitioners are actively facing, as observed in Betts *et al.*, 2024.

In a related InfoQ article, Hunter and Stojmilova, 2025 described a transformation journey at a software organization adopting Team Topologies and decentralized architectural decision-making. Their account illustrates several socio-technical dynamics that are directly relevant to this work: the initial disbelief of teams that they are actually empowered to make architectural decisions, the flood of decisions that follows once empowerment is accepted, the consequences teams face when decisions have unforeseen cross-boundary impacts, and the eventual self-correction as teams develop peer accountability mechanisms. All four phases are driven by social factors at least as much as technical ones.

A further grey-literature source relevant to this work is the InfoQ article by Tibbitts, 2025, which argues that "distributed architecture is more social than technical." Tibbitts draws on Andrew Harmel-Law's concept of the Architecture Advice Process — in which any team can make any architectural decision as long as it has looked for advice from the affected parties and documented the decision in an ADR. This approach is presented as a mechanism for replacing permission-seeking with advice-seeking. The social shift from "who approves this?" to "who should I talk to before we do this?" is presented as a cultural unlock that enables decentralized governance without sacrificing coherence.

3.3 Tool Support for Architectural Decision-Making

Along with the author's exploratory research, several tools and platforms have been proposed to support architectural decision-making:

- **ADR tools** (e.g., adr-tools, Log4brains) focus on recording and versioning Architecture Decision Records (ADRs). They capture the *what* and *why* of decisions but do not model uncertainty, impact, or technical plans for hypotheses that are not yet decisions. ADRs document settled choices; they are not designed to manage the pre-decision state of uncertainty that ArchHypo addresses.
- **Architecture analysis tools** (e.g., ATAM-based workshops, trade-off analysis methods) provide structured evaluation of architectural alternatives but are typically heavyweight, workshop-based activities rather than continuous, team-embedded processes. Their value lies in one-shot or periodic analysis sessions, not in the

ongoing, iterative management of uncertainty throughout a product's lifecycle.

- **Internal Developer Portals** such as Backstage, Port, and Cortex offer a catalog-centric view of services and infrastructure. While they improve discoverability and documentation, none of these platforms natively supports hypothesis engineering or architectural uncertainty management. They provide the integration surface that Hypo-Stage exploits, but leave the hypothesis engineering layer entirely absent.

Hypo-Stage differentiates itself by being explicitly designed around the ArchHypo lifecycle: from hypothesis elicitation through assessment, technical planning, and resolution. Its integration with Backstage places this lifecycle within the developer's daily tooling environment. No existing tool, to the best of the author's knowledge, provides this combination.

Tool / Platform	Primary Purpose	Models Uncertainty	Technical Plans	Lifecycle Tracking	Team-Embedded
ADR tools (adr-tools, Log4brains)	Record settled decisions.	No	No	Partial	No
ATAM / Trade-off workshops	Evaluate architectural alternatives.	Partial	No	No	No
IDPs (Backstage, Port, Cortex)	Developer portal; catalog; documentation.	No	No	No	Yes
Hypo-Stage (this work)	Support ArchHypo hypothesis engineering.	Yes	Yes	Yes	Yes (via Backstage)

Table 3.1: *Comparison of tools relevant to architectural decision-making support.*

3.4 Empirical Studies of Architectural Practices in Industry

Industry-focused empirical studies on architectural practices remain relatively scarce compared with tool proposals and method descriptions. Among the most relevant:

- The ArchHypo evaluations by Silva, Melegati, Silveira, *et al.*, 2025; Silva, Melegati, Wang, *et al.*, 2024 constitute the closest antecedents to this work. They demonstrated the value of hypothesis engineering in two distinct settings — a web-testing startup and a mission-critical government budget system — but relied entirely on manual application with no tool support. This work introduces the first evaluation of ArchHypo in a tool-mediated setting.

- Studies on architectural governance in agile organizations (e.g., the work on SAFe and Spotify-model variants) document how companies structure roles and ceremonies for architecture, but rarely examine explicit uncertainty-management techniques.
- The multivocal review conducted during the author's exploratory research surfaced practitioner reports (blog posts, conference talks, white papers) describing ad-hoc approaches to architectural uncertainty. These reports corroborate the need for structured methods but, being grey literature, carry weaker evidential value.

Based on the findings of these related studies, this present work contributes to the body of empirical evidence by evaluating a tool-supported application of ArchHypo with multiple teams, addressing a gap in both the ArchHypo literature (which has not yet studied tool-mediated use) and the broader architectural-practices literature (which lacks studies of hypothesis-engineering tools in scaled settings).

Chapter 4

Research Design

This chapter describes and justifies the research design adopted in this work. It covers the adopted research strategy along with the conceptual framework created to fullfill the research questions. Moreover, it covers the study context and case selection, the researcher's role, the data collection procedures, and the criteria adopted to establish the trustworthiness of the findings. The chapter concludes with notes on the ethical considerations that governed the study.

4.1 Research Strategy

The objective of this work was to empirically evaluate and refine Hypo-Stage, a support tool for the ArchHypo technique, by applying it with multiple software engineering teams in a real-world organizational setting. Answering the research questions posed in Chapter 1 requires understanding not only *what* outcomes the tool produces, but also *how* teams interact with it, *why* certain difficulties arise, and *which* adjustments improve its fit with existing practices. Following the design framework proposed by Robson and McCartan, 2016, the purpose of this work combined with its research questions clearly pointed out the need for a **flexible research design**.

Within flexible designs, the study follows a **multiple case study strategy** as defined by Robson and McCartan, 2016; Yin, 2009. A case study is defined by them as an empirical investigation of a particular contemporary phenomenon within its real-life context, using multiple sources of evidence and where the boundary between phenomenon and context is not clearly distinct. This definition fits the present work precisely: the phenomenon under study is the use of Hypo-Stage by a software engineering team, and its context — the organizational, technical, and social environment in which the team operates — is both relevant and inseparable from the phenomenon itself. A purely experimental design, which would require controlled manipulation of variables and prespecification of the design before data collection, is not appropriate here. The number of teams is small, the settings are naturalistic and non-manipulable, and the research purpose calls for rich, process-oriented accounts rather than statistical inference Robson and McCartan, 2016.

The study involves **two cases**, each consisting of a distinct software engineering team.

Using multiple cases rather than a single case follows the logic of *analytic (or theoretical) generalization* from Robson and McCartan, 2016; Yin, 2009: the two cases are not selected to constitute a statistical sample of teams, but to provide complementary and contrasting instances through which patterns of tool use, difficulties, and refinements can be examined and cross-validated. As Robson and McCartan, 2016 argues, multiple case studies are analogous to multiple experiments where each case either replicates findings or provides theoretically predicted contrast — both strengthening the analytic basis for generalization.

Beyond its descriptive and explanatory purposes, the study also has an **evaluative purpose**: it seeks to assess the worth and fitness of Hypo-Stage as a practical tool for managing architectural uncertainty in software engineering teams. Robson and McCartan, 2016 distinguish between *formative evaluation*, which aims at understanding and improving an ongoing program or intervention, and *summative evaluation*, which renders a final verdict on overall effectiveness. This study is explicitly formative: data collected during the observation period are used to identify difficulties and trigger tool refinements in near real-time, rather than to produce a retrospective pass-or-fail judgment.

Finally, the iterative loop through which the researcher observes difficulties, implements refinements to the tool, deploys an updated version, and resumes observation incorporates a core element of **action research** as described by Robson and McCartan, 2016: a deliberate cycle of plan, act, observe, and reflect, with the explicit intent of producing practical change alongside research knowledge. This does not transform the study into a full action research project — the primary goal remains the production of knowledge about tool use and architectural uncertainty, not organizational change more broadly — but the iterative refinement component follows the same logic and must be acknowledged as part of the design.

In summary, the research strategy can be formally described as a **flexible, multiple case study design with formative evaluation purpose and action research elements**, following the taxonomy from Robson and McCartan, 2016.

4.2 Conceptual Research Framework

The conceptual research framework organizes the key manufactures of this work and their relationships in a way that is consistent with the research questions and guides the data collection and analysis as proposed by Robson and McCartan, 2016. Figure 4.1 describes the framework.

The central piece is the *Hypo-Stage Tool Usage*, understood as the practices through which team members appropriate and practice the ArchHypo technique via Hypo-Stage: creating and editing architectural hypotheses, assessing uncertainty levels and impact, defining and tracking technical plans, and revisiting hypotheses over successive sprint cycles Silva, Melegati, Silveira, *et al.*, 2025. The tool usage is conditioned by two sets of contextual factors: the *team context* (composition, experience profile, scope of work, and familiarity with architectural concerns) and the *organizational context* (the company's development platform, the role of Backstage as the delivery portal, and the scaled product being developed). The tool usage produces two observable streams: the *artifacts* recorded in Hypo-Stage (hypotheses, assessments, technical plans, and their evolution over time)

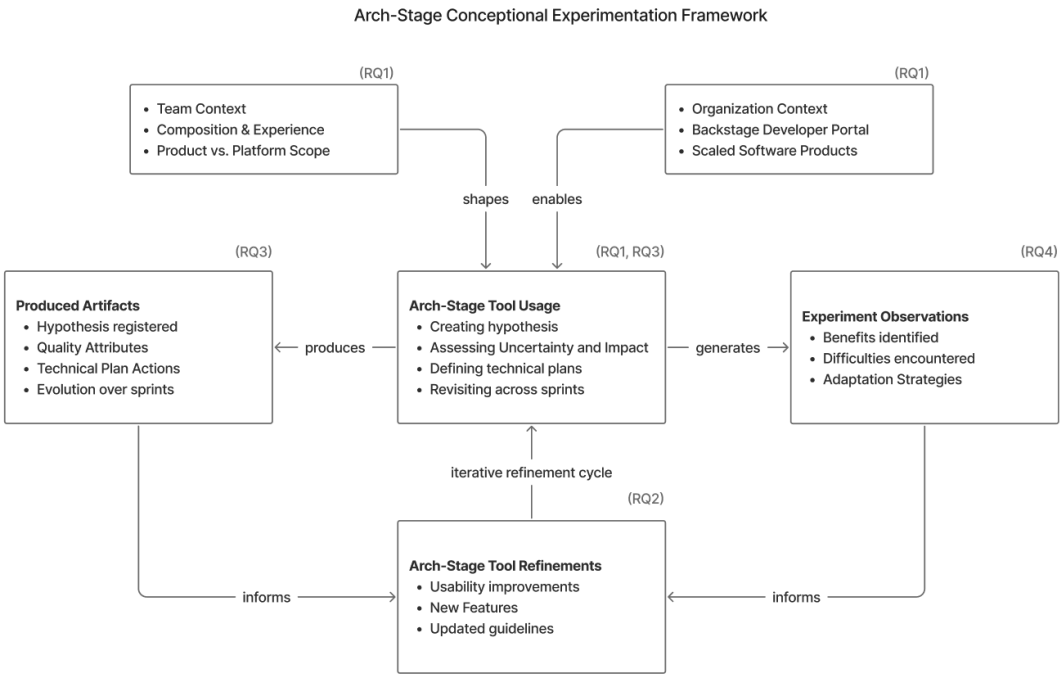

Figure 4.1: *Conceptual framework of the study.*

and the *experiment observations* of team members (benefits, difficulties, and adaptation strategies). These streams feed the *Hypo-Stage Tool Refinements* — changes made to Hypo-Stage and its usage guidelines — which in turn modify subsequent tool usage, forming an iterative cycle. This framework is consistent with the formative evaluation and action research elements of the design, and directly links each element to the research questions (RQ1–RQ4) stated in Chapter 1.

4.3 Research Context

The study was conducted within a technology organization that operates large-scale software products across Latin America and employs over 500 software engineers. The organization is an intensive user of Backstage,[1] an open-source developer portal, which serves as the central platform for internal tooling, documentation, and engineering workflows. The availability of this platform was an enabling condition for the study: Hypo-Stage was deployed as a Backstage plugin, so its adoption is conditioned by the availability of that platform. The Hypo-Stage tool was deployed directly in the company's production environment, integrated with the same catalog and workflow interface that engineers use daily, reducing adoption friction considerably. This allowed the researcher to observe the tool usage in a naturalistic setting, with minimal interference from the researcher.

[1] https://github.com/backstage/backstage

4.3.1 Case Selection

Two teams were selected as cases. Following Robson and McCartan, 2016's discussion of purposive sampling in flexible designs, the cases were not chosen to be statistically representative of a population of teams, but to provide *complementary contrast*: teams with different compositions, scopes, and relationships to the broader system, so that differences and similarities in tool use can be analytically compared.

Team 1 — Product Team. A team of 12 software engineers (5 senior, 7 mid-level) responsible for developing new product features with direct impact on the company's business objectives and end-user experience. This team works on a bounded product domain, with close proximity to business requirements and user feedback. Its architectural concerns are primarily about supporting new features reliably and at scale while preserving existing quality attributes.

Team 2 — Platform Team. A team of 4 software engineers (2 senior, 2 mid-level) responsible for developing and maintaining broad-scope internal tooling that supports software development across the entire organization. This team's architectural decisions affect how all engineers in the company build, deploy, and operate software. Its architectural concerns are therefore more horizontal, with higher impact on organizational engineering practices and greater uncertainty about usage patterns and requirements.

The two cases represent a *literal replication* in so far as both teams face the same intervention (use of Hypo-Stage and ArchHypo) and a *theoretical replication* in that their different scopes and compositions should, theoretically, lead to different experiences of architectural uncertainty and different challenges in adopting a hypothesis-driven approach Robson and McCartan, 2016; Yin, 2009. This contrast strengthens the analytic basis for identifying which findings are context-independent and which are conditioned by team type.

4.4 Researcher's Role

Throughout the study, the researcher occupied the dual role of **participant observer and facilitator**, in the terms described by Robson and McCartan, 2016. The researcher is an employee of the organization in which the study was conducted, which makes this an instance of *insider research*. This position provided privileged access to the teams, their work context, and their daily practices, which would have been substantially harder to obtain as an external investigator. It also allowed the researcher to introduce Hypo-Stage into the company's infrastructure in a credible and practically feasible way.

Robson and McCartan, 2016 acknowledge that insider research raises specific concerns regarding bias and the potential conflation of the researcher's own perspective with those of participants. These concerns are addressed in Section 4.8. It is important to note that, while the researcher served as facilitator during training sessions and provided assistance on request throughout the observation period, he did not direct the teams in *which* architectural uncertainties to address or *how* to formulate specific hypotheses. The content of the work produced by the teams in Hypo-Stage reflects their own judgments

Dimension	Team 1 — Product Team	Team 2 — Platform Team
Size	12 engineers (5 senior, 7 mid-level).	4 engineers (2 senior, 2 mid-level).
Scope	Bounded product domain; new features with direct business and user impact.	Broad-scope internal tooling supporting all engineers in the organization.
Architectural concerns	Supporting new features reliably at scale; preserving existing quality attributes.	Horizontal decisions affecting build, deploy, and operate practices organization-wide.
Uncertainty profile (expected)	Moderate uncertainty, high proximity to user feedback.	Higher uncertainty about usage patterns; high cross-team impact of decisions.
Replication logic	Literal replication — same intervention (Hypo-Stage + ArchHypo).	Theoretical replication — different scope and composition expected to surface different challenges.

Table 4.1: *Case comparison: Team 1 (Product) and Team 2 (Platform).*

and priorities.

4.5 Procedure

4.5.1 Training and Introduction

Before the observation period began, each team participated in a one-hour training session led by the researcher. The session covered:

(i) The conceptual foundations of hypotheses engineering and the ArchHypo technique, including the distinction between architectural hypotheses, uncertainty levels, impact assessment, and technical plans.

(ii) The context and motivation of the research, including the broader challenge of making architectural uncertainties explicit in teams working on scaled products.

(iii) A live demonstration of Hypo-Stage running in the company's Backstage instance, illustrating how to create and manage hypotheses within the tool. Teams were given written communication prior to the meeting (Appendix A.4) — including an illustration of the ArchHypo big picture (Figure 4.2) — so that they could arrive at the session with some prior exposure to the key concepts.

4.5.2 Observation Period

Following the training session, each team entered a two-sprint observation period of approximately four weeks, during which they were invited to use Hypo-Stage to register

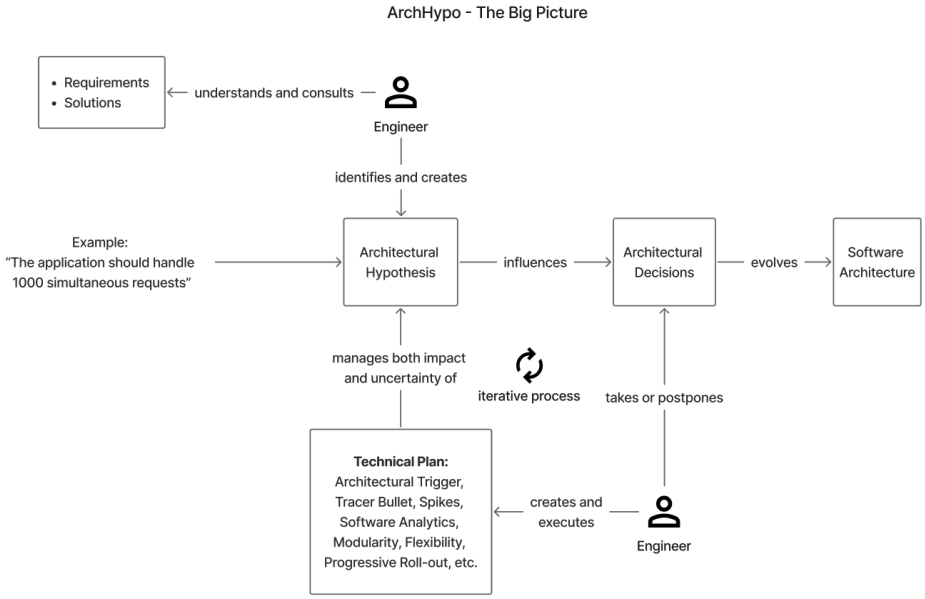

Figure 4.2: *Overview of the ArchHypo technique showing the iterative cycle through which engineers identify architectural hypotheses, assess their uncertainty and impact, define technical plans, and take or postpone architectural decisions. Source: adapted from Silva, Melegati, Silveira, et al., 2025.*

and manage any architectural uncertainties that arose in the course of their regular development work. No constraints were imposed on the type, number, or formulation of hypotheses, nor on when to create them during the sprints. It is intrisic to the nature of the organization in which the experience was conducted that the teams are free to adjust their agile practices to their own needs. Thefore, in contrast to Silva, Melegati, Silveira, *et al.*, 2025, the teams were not required to create hypotheses during the sprints, but rather to use the tools whenever they deemed necessary. The objective was to observe tool use in naturalistic conditions, with minimal researcher interference in the content of the teams' work.

During this period, the researcher remained available to both teams for support on request. Support was of two distinct types:

1. **Conceptual support.** When a team member found it difficult to formulate a hypothesis or to assess its uncertainty and impact within the ArchHypo framework, the researcher provided clarification or facilitated a brief discussion. Interactions of this type were noted as observations about the conceptual difficulty of the technique.

2. **Tool support.** When a team member encountered a difficulty that originated from the tool itself — a usability problem, a missing feature, or a mismatch between the tool's behavior and the expected workflow — the researcher logged this as a tool improvement candidate and, as a matter of priority, implemented and deployed an updated version of Hypo-Stage as quickly as practicable. This was done to ensure that tool limitations would not persistently obstruct teams' ability to practice ArchHypo, and to allow subsequent observations to reflect a more refined version of the tool.

This second type of support operationalizes the *action research cycle* within the study:

difficulty observed → refinement designed → updated tool deployed → use and observation resumed. Each refinement cycle is documented in Section 5.4.

4.6 Data Collection

Instrument	Data Produced	Collection Timing	RQs Addressed
Participant observation (field notes)	Reactions, use patterns, informal commentary, support requests.	Continuous during observation period.	RQ1, RQ3
Artifact analysis (Hypo-Stage records)	Hypotheses, assessments, technical plans, lifecycle transitions.	End of observation period.	RQ1, RQ2
Open-ended active listening sessions	Perceived usefulness, difficulties, suggestions, overall assessment.	End of observation period.	RQ3, RQ4

Table 4.2: *Data collection instruments, sources, and research questions addressed.*

Following the multiple data source approach recommended for case studies by Robson and McCartan, 2016; Yin, 2009, data were collected through three complementary channels:

4.6.1 Participant Observation

The researcher's continuous presence in the organization and involvement in both teams' environments provided an ongoing stream of observational data. Observations covered:

(i) Team members' reactions and difficulties during and after the training session;

(ii) Patterns of use (or non-use) of Hypo-Stage during the sprint;

(iii) Interactions in which team members requested conceptual or tool support;

(iv) Informal commentary about the fit of ArchHypo with existing practices.

Following Robson and McCartan, 2016's guidance on participant observation, field notes were recorded as close as possible in time to the observed events, and the researcher maintained a reflexive awareness of his dual role as insider and investigator.

4.6.2 Artifact Analysis

The hypotheses, assessments, and technical plans created by each team in Hypo-Stage constitute a documentary record of how they used the tool over the observation period. These artifacts were collected at the end of the observation period and analysed to characterize:

(i) The number and types of hypotheses created;

(ii) The quality attributes and sources of uncertainty they addressed;

(iii) How hypotheses were assessed and what technical plan actions were defined;

(iv) How hypotheses evolved (created, refined, or closed) across the two sprints.

The content analysis of documentary record is a recognized data collection method in flexible case study designs by Robson and McCartan, 2016.

4.6.3 Open-Ended Active Listening Sessions

In addition to informal interactions arising from support requests, the researcher conducted brief open-ended active listening sessions with team members at the end of the observation period, inviting them to reflect on their experience of using Hypo-Stage and ArchHypo. These conversations followed a semi-structured format, covering the perceived usefulness of the technique, the difficulties encountered, the changes they would suggest to the tool or its integration into their workflow, and their overall assessment of the value of making architectural uncertainties explicit in this way. Semi-structured or focused interviews are a central method for flexible design research and are particularly appropriate when the researcher seeks to capture participants' perspectives and interpretations rather than confirming predetermined hypotheses as described by Robson and McCartan, 2016.

4.7 Data Analysis

Data from the three collection sources were analysed using **thematic coding analysis**, following the procedure described by Robson and McCartan, 2016 for qualitative data in flexible designs Robson and McCartan, 2016. The process proceeded in three stages.

First, all field notes, artifact summaries, and active listening session records were reviewed and coded inductively, generating an initial set of descriptive codes capturing recurring observations and themes. Second, codes were grouped into higher-level categories corresponding to the constructs in the conceptual framework: *patterns of tool use*, *types of architectural uncertainty managed*, *perceived benefits*, *perceived challenges*, and *candidate tool refinements*. Third, patterns within and across the two cases were compared to identify what was common to both teams and what appeared specific to the product or platform context.

The artifact data from Hypo-Stage were also summarized descriptively (number of hypotheses per team, quality attributes addressed, evolution across sprints) to complement and illustrate the qualitative findings. The use of these summary statistics is consistent with Robson and McCartan, 2016's observation that flexible designs can include small amounts of quantitative data without becoming multistrategy designs.

Criterion	Meaning	Strategy Applied
Credibility	Internal validity; do findings accurately represent participants' realities?	Prolonged engagement (4 weeks); triangulation across three data sources; member checking.
Transferability	Degree to which findings may inform other settings.	Thick description of context; analytic (not statistical) generalization claimed.
Dependability	Reliability and consistency of the research process.	Audit trail: dated field notes, systematic artifact extraction, commits linked to each refinement.
Confirmability	Degree to which findings reflect data rather than researcher bias.	Explicit role separation; active search for disconfirming evidence; all claims grounded in specific observations or artifacts.

Table 4.3: *Trustworthiness criteria and strategies applied in this study (Robson and McCartan, 2016).*

4.8 Establishing Trustworthiness

The trustworthiness of findings from flexible design research is discussed by Robson and McCartan, 2016 in terms of *credibility*, *transferability*, *dependability*, and *confirmability* Robson and McCartan, 2016. Each is addressed below.

Credibility — the internal validity of the findings — was supported through:

(i) Prolonged engagement with both teams over four weeks of real work in their natural environment;

(ii) Multiple data sources (observation, artifact analysis, active listening sessions), providing triangulation of evidence;

(iii) Member checking, in which interpretations derived from observations and conversations were shared with team members for verification and correction.

Transferability — the degree to which the findings may inform other settings — is constrained by the specifics of the organizational context (a large technology company in Latin America with an established Backstage platform) and the tool (Hypo-Stage). The study does not claim statistical generalization. Rather, it offers analytic generalization Robson and McCartan, 2016; Yin, 2009: the patterns of use, difficulties, and refinements documented here provide a theoretically informed account that researchers and practitioners can assess for relevance to their own contexts.

Dependability — the reliability of the research process — was addressed by maintaining an audit trail: field notes were dated and linked to specific events, artifact data were extracted and stored systematically, and each tool refinement was documented with its triggering observation and the corresponding commit in the Hypo-Stage repository.

Confirmability — the degree to which findings reflect the data rather than researcher bias — was addressed by:

(i) Explicitly distinguishing between the researcher's facilitation role (which involved helping teams understand the technique) and his investigator role (which required non-directive observation of how they used it);

(ii) Seeking disconfirming evidence, that is, noting instances where teams found value in the tool or the technique as readily as instances of difficulty;

(iii) Grounding all claims in specific observations, artifacts, or conversation excerpts, reported in Chapter 5 (including Section 5.4 where refinements are documented).

4.9 Ethical Considerations

The study raises ethical considerations associated with insider research, as discussed by Robson and McCartan, 2016. Participation was voluntary: team members were informed of the research objectives, were free to use Hypo-Stage as much or as little as they chose, and were invited but not required to engage in active listening sessions. No personal performance data were collected or reported; all analysis pertains to team-level patterns and tool-level observations. The organization in which the study was conducted and the teams involved are anonymized in reporting. As no sensitive personal data were collected, no ethical review was required for this study.

The dual role of the researcher as organizational employee and academic investigator is a potential source of role conflict. This was managed by maintaining a clear separation between the researcher's responsibilities to the organization (deploying and supporting a useful tool) and his responsibilities as a researcher (reporting findings honestly, including negative ones). The research was conducted with the knowledge and informal consent of the relevant engineering leadership in the organization.

Chapter 5

Results and Refinements

This chapter presents the empirical findings of the study and the refinements to Hypo-Stage that emerged from them. The results are organized around the four research questions defined in Chapter 1: RQ1 addresses how teams used the tool and the technique in practice; RQ2 characterises the architectural uncertainties they registered; RQ3 reports the benefits and challenges perceived by participants; and RQ4 documents each refinement, tracing it to a concrete observation and to the question it addresses. Section 5.5 goes beyond a direct answer to the four questions by drawing two families of *emerging patterns*: team-behaviour patterns (Group A) and a set of novel technical architectural patterns for hypothesis engineers (Group B). The latter constitutes a distinct contribution of this work, addressing a gap left open by prior research in the ArchHypo line, as discussed in Section 5.5.2.

5.1 RQ1 — How Teams Used Hypo-Stage in Practice

5.1.1 Hypothesis Registration Dynamics

Over the four-week observation period, both teams together registered thirteen architectural hypotheses in Hypo-Stage. Six hypotheses originated from the Product Team and seven from the Platform Team. This distribution is noteworthy because the Platform Team is significantly smaller (four engineers) yet produced more registered hypotheses than the twelve-engineer Product Team. This asymmetry is partly explained by the registration of a dense cluster of five Platform hypotheses on a single day (12 April 2026), all addressing complementary aspects of the shared Kafka streaming infrastructure — a burst pattern discussed further in Section 5.5.2.

Across both teams, twelve of the thirteen hypotheses were authored by senior technical leaders — staff or principal engineers who held explicit architectural responsibilities for their respective domains. One hypothesis was registered by a mid-level engineer on the Product Team. Both technical leaders in each team independently reported that a much larger number of architectural uncertainties existed in their daily work than the number they managed to register. The principal constraint they named was time: formulating

a well-structured hypothesis statement requires focused, uninterrupted attention that sprint delivery pressure rarely afforded. This observation resonates with findings from prior ArchHypo evaluations, where the learning curve and process-integration demands were described as significant barriers to adoption Silva, Melegati, Silveira, *et al.*, 2025; Silva, Melegati, Wang, *et al.*, 2024.

5.1.2 Role-Based Engagement

Participant observation and the active listening sessions revealed a clear stratification in how different roles engaged with Hypo-Stage during the study period. Senior engineers treated the tool as a decision-support instrument: they used it to externalise concerns that had until then existed only as tacit knowledge, and they referenced the hypothesis backlog when discussing architectural trade-offs in planning sessions. Mid-level and junior engineers were aware of the tool's existence and attended the training session, but neither team saw them register hypotheses independently beyond the initial introduction. This does not indicate disengagement; rather, both technical leaders explained that junior engineers typically operate at a scope where architectural uncertainties are resolved quickly — within a sprint or less — leaving little perceived need for formal registration.

5.1.3 Tool Adoption Patterns

Usage of Hypo-Stage was non-uniform across the observation window. Registration activity was concentrated in two periods: a first wave during and immediately after the training session, and a second, more intensive wave near the end of the period when the Platform Team used the tool for a structured architectural discussion on Kafka failure-handling strategies. Between these waves, both technical leaders reported using the hypothesis detail view to review existing records and update technical plans, but the tool was not used in team-visible ceremonies such as sprint retrospectives or planning meetings. When asked, participants noted that embedding the tool into these ceremonies would require an explicit cultural agreement within the team — something that had not yet occurred within the observation window.

The integration of Hypo-Stage into the Backstage Internal Developer Portal was cited by both teams as a meaningful enabler of adoption. Because Backstage is already the daily entry point for service catalogue, documentation, and CI/CD status, engineers encountered Hypo-Stage without navigating to a separate tool. The EntityHypothesesTab, which surfaces only the hypotheses linked to the component an engineer is already viewing, was specifically mentioned as reducing the cognitive cost of context-switching into hypothesis management.

5.2 RQ2 — Architectural Uncertainties Registered

5.2.1 Data Overview

Table 5.1 presents the thirteen hypotheses registered during the study, anonymised in accordance with the ethical agreement described in Section 4.9. Full exported records

(statements, ratings, entity links, and author notes) appear in Appendix B. Each hypothesis is identified by a short label, its source type (Requirements or Solution), its lifecycle status at the end of the observation period, and its uncertainty and impact ratings.

ID	Short label	Team	Source	U	I
H01	Kotlin/JOOQ/Spring template readiness	Product	Req.	VH	VH
H02	Kafka direct exposure via platform framework	Platform	Req.	VL	M
H03	Decouple eng. logs from security/compliance audit	Platform	Req.	M	VH
H04	Blob storage GDPR right-to-erasure	Product	Req.	M	VL
H05	Credit-bureau decoupling seam	Product	Sol.	H	VH
H06	Billing aggregation chunked keyset pagination	Product	Sol.	VL	VH
H07	Blob ownership transfer across service domains	Product	Req.	VL	VH
H08	Blob cross-client sharing governance	Product	Req.	VL	VH
H09	DLT for audit vs. offset-based replay	Platform	Sol.	M	VH
H10	Synchronous retry for transient consumer failures	Platform	Sol.	M	H
H11	Failure state inside vs. outside Kafka flow	Platform	Sol.	M	H
H12	Error classification before retry	Platform	Sol.	M	H
H13	Payload immutability before re-enqueueing	Platform	Sol.	VL	VH

Table 5.1: *Overview of the thirteen registered architectural hypotheses. Req. = Requirements-sourced; Sol. = Solution-sourced. U = Uncertainty level; I = Impact level. Ratings: VL = Very Low, M = Medium, H = High, VH = Very High.*

5.2.2 Source Types

Eight of the thirteen hypotheses were classified as *Requirements*-sourced: the uncertainty stems from an incompletely specified quality-attribute goal or an external obligation whose implementation path is unknown. Five were classified as *Solution*-sourced: the team has an existing or candidate architectural decision and is uncertain about its consequences. The predominance of requirements-sourced hypotheses on the Product Team (five out of six) and the predominance of solution-sourced hypotheses on the Platform Team's Kafka cluster (four out of five Kafka hypotheses) reflects the different nature of each team's work: the Product Team faces open questions about whether architectural goals can be met at all, while the Platform Team is assessing the trade-offs of specific implementation choices for a shared infrastructure component.

5.2.3 Quality Attribute Distribution

Table 5.2 summarises the frequency of quality attributes across the thirteen hypotheses. Attributes were assigned by the hypothesis authors using the multi-select form in HypoStage; a hypothesis may carry more than one attribute.

Reliability and Auditability emerge as the two dominant concerns. The prominence of Auditability is especially notable and directly traceable to two contextual forces: the organization's internal security audit requirements, which drove H03, and the applicable privacy regulation governing binary document storage (H04), as well as the Kafka immutability and DLT governance questions (H09, H13). This concentration reveals a

Quality Attribute	Count	% of hypotheses
Reliability	6	46%
Auditability	5	38%
Performance	4	31%
Interoperability	4	31%
Extensibility	3	23%
Usability	3	23%
Scalability	2	15%
Modifiability	2	15%
Flexibility	2	15%
Configurability	2	15%
Maintainability	1	8%
Security	1	8%
Observability	1	8%
Reusability	1	8%

Table 5.2: *Quality attribute frequency across the thirteen hypotheses.*

recurring archetype in which external compliance obligations crystallise into architectural hypotheses — a pattern discussed in Section 5.5.2 as the *Regulatory Trigger Hypothesis*.

5.2.4 Uncertainty and Impact Profile

Nine of the thirteen hypotheses carry a *Very High* or *High* impact rating, signalling that the teams were not registering peripheral concerns but uncertainties with potentially significant consequences for schedule, rework, or system quality. Uncertainty ratings are more evenly distributed: five hypotheses are rated *Very Low* (the team has strong evidence but the hypothesis is tracked for governance reasons), four are *Medium*, one is *High*, and one *Very High*.

The combination of Very Low uncertainty and Very High impact (H06, H07, H08, H13) is particularly instructive. In ArchHypo's model, this quadrant maps to hypotheses that *can be postponed* — the team already has sufficient evidence to act, so the immediate risk is low — yet the downstream consequences of acting incorrectly are severe. Hypo-Stage's prioritisation dashboard surfaced these hypotheses in the "Can Postpone" segment of its focus panel, enabling teams to consciously defer them without losing visibility. This represents a practical instance of the *Postpone Uncertain Decisions* pattern described in Silva *et al.* (2024).

5.2.5 Technical Plans in Use

Seven of the thirteen hypotheses had at least one associated technical plan at the end of the observation period. Together these seven hypotheses carried thirteen individual technical planning entries. The complete catalog of plan titles and strategies is reproduced in Section B.3 of Appendix B. Table 5.3 summarises the strategies used.

Guideline Implementation and Architectural Spikes were the most frequently chosen

Strategy	Plans	Hypotheses using it
Guideline Implementation	3	3
Spike (Architectural Spike)	3	2
Modularisation / Flexibility	2	1
Experiment	2	2
Tracer Bullet	1	1
Software Analytics (Trigger)	1	1
Other (implementation)	1	1
Total	**13**	—

Table 5.3: *Technical plan strategies observed across the hypothesis data.*

strategies, consistent with findings from the PropSEP/Jetsoft evaluation (Silva, Melegati, Silveira, *et al.*, 2025). The Tracer Bullet strategy appeared once — for H10, where the Platform Team planned a controlled injection of transient failures into a real consumer path to measure ordering and latency characteristics. This application illustrates how the Tracer Bullet differs from a Spike: rather than an isolated experiment, the plan exercises the full production consumer path under representative conditions (Silva, Melegati, Silveira, *et al.*, 2025).

Six hypotheses had no technical plan at the end of the study. Three of these (H01, H04, H02) were registered but not yet acted upon; two (H07, H08) had been validated through planning actions that predated the observation window and whose plans were registered retrospectively; and one (H05) was under active discovery but the plan document was still being prepared.

5.2.6 Lifecycle and Status

At the end of the observation period, ten hypotheses had *Open* status, one was *In Review* (H05, the credit-bureau decoupling, which had moved to a structured discovery phase), and two were *Validated* (H07 and H08, both in the blob storage domain). No hypothesis was invalidated or discarded during the study window. The predominance of the Open status is consistent with the short duration of the study (four weeks, two agile sprints) and with the nature of the uncertainties: most concern mid- to long-horizon architectural concerns that teams do not expect to resolve within a single sprint cycle.

5.3 RQ3 — Perceived Benefits and Challenges

5.3.1 Perceived Benefits

Active listening sessions with all four interviewed participants (the two technical leaders from each team) converged on three recurring themes regarding the perceived value of using Hypo-Stage.

Externalisation of tacit architectural knowledge. Both teams described the act of registering a hypothesis as valuable independent of what the tool subsequently did with the record. Articulating an uncertainty in falsifiable form forced a degree of precision that normal sprint conversations do not require. Participants noted that seeing a hypothesis written down — with an explicit uncertainty rating, an impact rating, and a related quality attribute — made it easier to share the concern with colleagues who had not been involved in the original discussion.

Prioritisation support. The *Where to Focus* panel of the Hypo-Stage dashboard — distinguishing hypotheses that *Need Attention* (high uncertainty and high impact) from those that *Can Postpone* (low impact) — was described by both technical leaders as a useful aid in architectural conversations. One participant noted that the "Can Postpone" category in particular gave them a principled way to explain to product managers why a known concern was being deliberately deferred rather than ignored.

Collective memory for architectural decisions. Both teams observed that the hypothesis backlog served a documentation function that was distinct from Architecture Decision Records (ADRs) or design documents. Whereas ADRs capture decisions after they are made, an architectural hypothesis captures an open question before a decision is available. Participants described this as filling a gap in their current toolchain: they had good mechanisms for recording what was decided but poor mechanisms for recording what was still unknown.

5.3.2 Perceived Challenges

Three challenges were consistently raised across participants.

Negative-formulation learning curve. The convention of writing hypotheses as falsifiable statements — phrased in a form that specifies under which evidence the team would conclude the hypothesis is incorrect — was described by all four participants as non-intuitive. Engineers are accustomed to stating what they believe to be true. Inverting that belief into a falsifiable form that describes what would prove it false requires a deliberate cognitive effort that is easy to skip under time pressure. This finding replicates the learning curve reported in both prior ArchHypo evaluations (Silva, Melegati, Silveira, *et al.*, 2025; Silva, Melegati, Wang, *et al.*, 2024).

Registration time cost. Both technical leaders described the difficulty of dedicating the focused time required to register a hypothesis properly. Fast-moving architectural micro-decisions — resolved within a day or less — were rarely registered because, by the time a hypothesis could be written, the question had already been answered informally. This creates a systematic under-representation of short-lifecycle uncertainties in the hypothesis backlog, which may be consequential when the same question re-emerges in a future context.

Absence of team-visible ceremonies. Neither team had established a recurring ceremony — such as an architectural review or a hypothesis triage meeting — in which

Hypo-Stage records were consulted as a team. Tool usage remained largely individual during the observation period. Both technical leaders expressed the view that realising the collaborative potential of hypothesis engineering would require the team to institutionalise at least one touchpoint where the backlog is reviewed together.

5.4 RQ4 — Refinements to Hypo-Stage

This section presents the refinements implemented in Hypo-Stage from commit `ceee509` (the artefact boundary established by Corrêa 2025) to commit `3a53d4e` (HEAD at the time of writing). Each refinement is traced to a concrete observation from the study and linked to the research question it primarily addresses. All refinements address RQ4 by definition; selected refinements also bear on RQ1 (adoption ease) and RQ3 (usability and learning curve).

5.4.1 Backstage Integration and Catalog Surfacing

Observation. During the training session and the first week of observation, both teams navigated to Hypo-Stage via a standalone entry point that was separate from their regular Backstage workflow. Engineers who were not technical leaders rarely revisited the tool after the initial session.

Refinement. Hypo-Stage was wired as a first-class Backstage experience, with routable pages for home, hypothesis creation, detail view, and edit flows. Crucially, a catalog component tab (`EntityHypothesesTab`) was introduced, surfacing all hypotheses linked to the component an engineer is currently viewing in the Backstage catalog. A feature flag (`hypo-stage`) was added to allow gradual, team-controlled rollout.

Impact. Reducing navigation friction lowered the perceived cost of checking the hypothesis backlog while already working with a service's catalog entry. The Platform Team's technical leader noted that the component tab made hypothesis review a natural side-effect of existing Backstage usage patterns.

5.4.2 Backend API and Filtering

Observation. Engineers with cross-team or platform-wide responsibilities needed to filter hypotheses by team and by specific service component. The initial version of Hypo-Stage returned all hypotheses without server-side filtering, which degraded usability as the hypothesis data grew.

Refinements. Three new API capabilities were implemented: (i) `GET /hypotheses` extended with `entityRef` and `team` query parameters; (ii) `GET /hypotheses/referenced-entity-refs` to populate the component filter without listing the entire catalog; and (iii) `GET /hypotheses/teams` deriving team names from catalog metadata for components referenced by hypotheses.

5.4.3 Prioritisation Dashboard

Observation. Both technical leaders expressed the desire to quickly identify which hypotheses deserved immediate attention and which could safely be deferred. Manually scanning the full hypothesis list imposed an unnecessary cognitive burden, especially when the backlog grew.

Refinement. A `HypothesisListDashboard` component was introduced above the hypothesis table, presenting: (i) overview cards with total count, validated count, and hypotheses created in the last thirty days; (ii) a *Where to Focus* section distinguishing *Need Attention* (high uncertainty and high impact) from *Can Postpone* (low impact), with explanatory subtext aligned with ArchHypo's prioritisation model (Silva, Melegati, Silveira, *et al.*, 2025); and (iii) distribution panels for uncertainty and impact Likert values. The dashboard respects entity, team, and component scoping.

Impact (RQ3). The "Can Postpone" segment was specifically cited by participants as resolving a communication challenge: it provided a principled vocabulary for explaining deliberate deferral to non-technical stakeholders.

5.4.4 Hypothesis Lifecycle Consistency

Observation. In the initial implementation, create and update operations were not wrapped in database transactions, which created a risk of partial writes (hypothesis row updated without the corresponding event entry, or vice versa). Artifact analysis revealed two records with missing event entries, suggesting that this race condition had occurred in practice.

Refinements. Create and update paths were refactored to use explicit database transactions. Delete was extended to cascade to technical planning rows and events, with clear API semantics and a confirmation dialog that requires the exact hypothesis statement before proceeding (paste-enabled to avoid penalising users with long statements). Success and error toast notifications were added for destructive operations.

5.4.5 Technical-Planning-Driven Assessment Updates

Observation. Engineers reported that after completing a technical plan action — such as an architectural spike — they updated the hypothesis uncertainty rating separately from the plan record. This two-step process was error-prone and led to hypotheses whose assessment was stale relative to their technical plans.

Refinements. Technical planning create and update operations were extended to accept optional `uncertainty` and `impact` fields. When present, the service updates the parent hypothesis's Likert values in the same transaction. The event model was refined to distinguish `TECHNICAL_PLANNING_CREATE` and `TECHNICAL_PLANNING_UPDATE` events from generic hypothesis updates, preserving a traceable audit trail of which assessment changes were driven by plan actions.

5.4.6 Evolution Chart Improvements

Observation. The initial uncertainty/impact evolution chart used opaque numeric y-axis ticks that participants could not interpret without consulting the Likert scale documentation separately. Multiple events on the same calendar day produced ambiguous x-axis labels.

Refinements. The chart was updated to render verbal y-axis ticks (Very Low through Very High). Event source was made visually distinguishable: circles for manual or creation-driven points, squares for technical-planning-driven points. A legend explaining the marker shapes was added. When multiple events occur on the same calendar day, the x-axis uses date and time to avoid ambiguity. Tooltips were improved to label technical-planning-driven assessment moves with the plan number when identifiable.

5.4.7 Form Usability Improvements

Observation. Participants found the hypothesis creation form cognitively demanding, particularly the uncertainty and impact selectors, which displayed only numeric values, and the quality attribute field, which accepted free text without suggesting valid values.

Refinements. Uncertainty and impact selectors were redesigned as Likert star pickers showing both a numeric button (1–5) and an adjacent text label (Very Low through Very High) with disambiguating icons. Quality attributes were replaced by a multi-select autocomplete showing attribute name and description in the dropdown. Entity references for linking hypotheses to catalog components were given a catalog-backed autocomplete. Documentation link fields were given an improved list editor with add/remove and basic URL validation.

5.4.8 Summary: Refinements Mapped to Observations

Table 5.4 summarises the nine categories of refinements, the primary observation that motivated each, and the research question each addresses most directly.

5.5 Emerging Patterns

Beyond the direct answers to RQ1–RQ4, thematic analysis of observation notes, active listening transcripts, and the combined hypothesis data produced two families of patterns. Group A describes team-behaviour conditions that shape whether hypothesis engineering becomes a sustainable practice. Group B describes recurring structural archetypes in the content of the architectural hypotheses themselves. These two families address orthogonal dimensions: Group A asks *when* hypothesis engineering works; Group B asks *what shape* hypotheses take in practice.

Refinement	Motivating observation	RQ1	RQ4	RQ3
Backstage catalog tab	Infrequent revisit by non-leaders outside native workflow	S	P	S
Backend API filtering	Cross-team users needed scoped views as hypothesis data grew	S	P	—
Prioritisation dashboard	Manual scanning of full backlog was cognitively costly	—	P	S
Lifecycle transaction consistency	Partial writes produced stale event records	—	P	—
TP-driven assessment updates	Assessment and plan records drifted apart post-spike	—	P	S
Evolution chart improvements	Numeric y-axis and same-day collisions caused confusion	—	P	S
Form usability improvements	Numeric-only selectors; free-text QA field caused errors	—	P	P
Safer deletion UX	Long statements made confirmation hostile without paste	—	P	—
Toast notifications	No feedback after destructive operations caused re-submission	—	P	—

Table 5.4: *Refinements to Hypo-Stage mapped to motivating observations and research questions. RQ columns indicate primary (P) or secondary (S) linkage.*

5.5.1 Group A — Team Behaviour Patterns for Hypothesis Engineering Adoption

The four Group A patterns emerge from triangulating RQ1 and RQ3 findings. They describe organizational and individual-level conditions that the study observed to facilitate or obstruct the adoption of hypothesis engineering as a team practice. These patterns complement the existing ArchHypo pattern languages (Silva *et al.*, 2024; Silva *et al.*, 2024), which focus on the mechanics of hypothesis assessment and deferral, by addressing the social and operational context in which those mechanics must operate.

A1 — Senior-Curated Hypothesis Backlog. **Problem.** How can a team ensure that mid- and long-horizon architectural uncertainties remain visible and actively managed over time, even though individual engineers work at different scopes and timescales?

Observed evidence. Twelve of the thirteen registered hypotheses were authored by senior technical leaders — engineers who carried explicit architectural responsibility and whose work horizon extended beyond the current sprint. Non-senior engineers did not sustain independent hypothesis registration after the initial training episode. The hypotheses that were registered shared a common trait: they concerned uncertainties whose consequences would materialise weeks or months in the future, making them invisible to typical sprint-scope review practices.

Solution. Assign senior engineers as the primary curators of the hypothesis backlog. Their architectural horizon and organizational authority position them to recognise which uncertainties are worth formalising. Non-senior participation should be enabled by the tool but cannot be assumed to be self-sustaining in the early phases of adoption.

A2 — Fast-Resolution Blind Spot. **Problem.** Many architectural micro-decisions are made and resolved within hours or a single day. Their speed makes them feel too transient to register, yet their cumulative architectural effect may only become visible — as drift or inconsistency — long after the original decision context is forgotten.

Observed evidence. Both technical leaders reported that the majority of architectural decisions encountered day-to-day were resolved before a hypothesis could be written. This speed made registration feel retrospectively unnecessary. Yet both also described recurring situations where the same question re-surfaced in a new context, requiring the team to reconstruct reasoning that had previously been resolved informally.

Solution. Introduce a lightweight periodic review — for example, a brief end-of-sprint ceremony — specifically to identify decisions made in the past iteration that resolved quickly but may carry cumulative consequences. Decisions with cross-component impact or quality-attribute implications are candidates for retroactive registration or lightweight decision records.

A3 — Registration as a Slack-Dependent Practice. **Problem.** Translating a tacit architectural concern into a properly formulated ArchHypo hypothesis requires focused, uninterrupted attention. Sprint delivery pressure makes this kind of attention scarce and unreliable as a basis for sustained practice.

Observed evidence. Senior leaders explicitly described the difficulty of "dedicating proper time to register all possible hypotheses they encounter day-to-day." The observed set of thirteen hypotheses is known to under-represent the total uncertainties that leaders were aware of. Registration activity clustered around protected discussion time — the training session and a structured team session on Kafka strategy — rather than being distributed across normal sprint work.

Solution. Treat hypothesis registration as a deliberate activity requiring protected time — a recurring architectural hygiene session, separate from sprint ceremonies, where engineers externalise concerns that have been accumulating in tacit knowledge. The IDP integration of Hypo-Stage reduces navigation cost but does not substitute for protected cognitive time.

A4 — Negative-Formulation Barrier. **Problem.** The ArchHypo convention of writing hypotheses as falsifiable statements — specifying the evidence that would prove the hypothesis false — is cognitively inverted relative to how engineers normally articulate beliefs. This inversion is a persistent adoption barrier even for practitioners who understand the technique conceptually.

Observed evidence. All four interviewed participants mentioned that the falsifiability requirement is non-intuitive and difficult to apply under time pressure. This replicates

findings reported in the PropSEP/Jetsoft evaluation (Silva, Melegati, Silveira, *et al.*, 2025) and remains unresolved in the current version of the technique's guidance materials.

Solution. Provide statement templates that scaffold the inversion: starting from an architectural belief ("We believe X is true") and guiding the engineer to express what observable outcome would prove it false. Embedding this scaffolding inline in the hypothesis creation form — as contextual guidance adjacent to the statement field — would reduce the cognitive overhead at the moment of registration without requiring prior training.

5.5.2 Group B — Technical Architectural Patterns for Hypothesis Engineers

The prior work in the ArchHypo research line has established the technique's core concepts (Silva, Melegati, Silveira, *et al.*, 2025), documented patterns for hypothesis assessment, uncertainty management, and decision deferral (Silva *et al.*, 2024; Silva *et al.*, 2024), and reported on the technique's learning curve and process-integration challenges (Silva, Melegati, Wang, *et al.*, 2024). What this body of work has not yet addressed is a question that only becomes answerable through empirical data on hypotheses registered in practice:

> *What recurring shapes do architectural hypotheses take when teams apply ArchHypo to real systems? And what structural properties of a hypothesis — its source type, quality-attribute profile, and relation to its organizational context — should an engineer recognise in order to write and manage it more effectively?*

This gap was also identified in calls for future work within the ArchHypo pattern literature, which explicitly noted the need for "identifying and documenting patterns for various types of hypotheses" and for "exploration of common uncertainties related to specific quality attributes" (Silva *et al.*, 2024). The thirteen-hypothesis dataset produced by this study constitutes the first empirical dataset that enables such identification. The five patterns described below are therefore a **novel contribution** of this work: they are grounded in real hypothesis data, not derived from theoretical analysis, and they provide a vocabulary that hypothesis engineers can use both to recognise familiar situations more quickly and to communicate about hypothesis types across teams.

B1 — Hypothesis Cluster. **Context.** A team is responsible for a shared platform component on which multiple other service teams depend. An architectural question about that component cannot be decomposed into a single hypothesis because the design space has several interrelated dimensions, each of which involves a distinct trade-off.

Problem. Formulating a single hypothesis for a multi-dimensional architectural concern either oversimplifies the problem (losing important trade-off structure) or produces an unmanageably broad statement that is hard to falsify.

Observed evidence. Five hypotheses (H09–H13) all concern the shared Kafka streaming infrastructure and were registered on the same day, as the Platform Team's technical leader used Hypo-Stage to structure a design discussion. The five hypotheses address: (i) DLT audit semantics versus offset-based replay; (ii) synchronous versus asynchronous

retry for transient consumer failures; (iii) failure-state placement inside versus outside the Kafka flow; (iv) error classification before retry; and (v) payload immutability before re-enqueueing. Each is independently falsifiable yet together they map the full decision space for consumer-side failure handling in the shared streaming platform.

Solution. When a shared platform component has architectural uncertainty involving multiple interrelated dimensions, decompose it into a cluster of related but independently falsifiable hypotheses. Assign the cluster to the same platform component in the tool so it appears together in the component-scoped tab. Manage technical plans at the individual hypothesis level but review the cluster as a unit in architectural discussions, as the resolution of one hypothesis may constrain the options available for others.

B2 — Regulatory Trigger Hypothesis. **Context.** A software system operates in a regulated domain — financial services, healthcare, data privacy — where external compliance obligations create architectural constraints whose implementation path is not fully known.

Problem. A compliance requirement imposes an architectural obligation before the team has determined how to satisfy it. It is unclear whether the existing architecture already satisfies the obligation, what changes would be required if it does not, and what the cost of non-compliance would be.

Observed evidence. Two hypotheses in the dataset arise directly from regulatory obligations: H04 (the GDPR right-to-erasure requirement for binary documents in the blob storage service) and H03 (organization-wide security audit requirements for database log correlation across the platform framework). In both cases the uncertainty is not about whether to comply — that is a legal given — but about whether the current architecture *can* comply and at what technical cost.

Solution. When a regulatory obligation creates an architectural constraint, formulate it as a hypothesis stating what the system must be capable of and making falsifiable the belief that the current architecture satisfies this capability. This transforms compliance work from an implicit assumption into an explicit tracked uncertainty with a technical plan and a validation target. The quality attributes Auditability and Security are diagnostic markers for this pattern.

B3 — Decoupling Seam Hypothesis. **Context.** A service currently handles multiple responsibilities — domain logic, external integrations, and identity management — in a tightly coupled implementation. The team believes the service could be restructured to serve multiple domain contexts, but the boundaries of the decoupling and the migration path are unknown.

Problem. Deciding *whether* to decouple is itself uncertain: the team does not know what the correct contract boundary should be, which existing flows can be migrated without business-logic changes, and what the downstream impact on other teams would be. Committing to decoupling without this information risks a disruptive refactor that delivers less reuse than expected.

Observed evidence. H05 (credit-bureau decoupling) captures this precisely. The service currently handles identity, bureau integration, and confidential document flows

using domain-specific entities tied to a single business vertical. The hypothesis asks whether these integrations can be decoupled into context-agnostic middleware accepting only each provider's minimum required identifiers, reusable across domains without duplicated cost. At the end of the observation period, H05 was the only hypothesis in *In Review* status, with a discovery technical plan mapping (i) the dependency matrix of current versus real dependencies per integration, (ii) an agnostic interface contract proposal, and (iii) a migration plan identifying which flows can isolate immediately.

Solution. When a team suspects that a service boundary is too narrow for the reuse it should enable, formulate the uncertainty as a decoupling seam hypothesis targeting the specific boundary in question. The technical plan should follow a discovery strategy before any implementation is attempted: map existing dependencies, propose an interface contract free of domain-specific concepts, and identify the migration sequence. Quality attributes Interoperability, Extensibility, and Reusability are diagnostic markers for this pattern.

B4 — Load-Validated Hypothesis. **Context.** A team has proposed a specific algorithmic or implementation approach to handle a high-volume data processing challenge. The team has already conducted a capacity or volume test at scale, producing evidence that the approach works. However, the hypothesis has not been formally closed because the implementation is still in progress.

Problem. The hypothesis occupies the lowest uncertainty quadrant (Very Low) while retaining the highest impact rating (Very High), reflecting a situation where the team *knows* the solution works but the architectural commitment is still consequential. The hypothesis must be tracked until implementation is complete, yet its characterisation as "Open" may suggest unresolved doubt that no longer exists.

Observed evidence. H06 (billing aggregation chunked keyset pagination) exemplifies this. The team validated the approach — chunked keyset pagination with isolated repository-level transactions — through a capacity test at five times production scale (45 million rows total, 1.5 million rows per month). Profiling confirmed that JPA stream processing caused first-level cache growth, and that chunking scopes transactions to approximately ten thousand rows so the garbage collector can reclaim memory, producing a healthy sawtooth heap pattern. The hypothesis remains Open because the implementation is ongoing, but the uncertainty has been effectively resolved.

Solution. When a team has strong empirical evidence that a solution approach is correct but implementation is incomplete, retain the hypothesis in Open status with a Very Low uncertainty rating and document the validation evidence in the hypothesis notes. Plan the hypothesis lifecycle so that it transitions to Validated upon completion of the production implementation, not upon completion of the test. The hypothesis note field should reference the capacity test artefact explicitly, preserving the evidentiary chain.

B5 — Shared Resource Governance Hypothesis. **Context.** A platform service manages a shared resource — binary objects, credentials, configuration data — on which multiple product domains depend. New business requirements demand that the resource be shared across domain boundaries or transferred between owning services, but the governance rules for these operations have not been established.

Problem. The team must decide how to model cross-domain access to the shared resource — specifically, which service "owns" the resource at any given point, who may read or write it, and under what conditions ownership can be transferred. These questions have significant security, consistency, and usability implications. Without explicit governance decisions, different product teams will adopt incompatible access patterns, creating consistency violations that are difficult to detect and remedy.

Observed evidence. H07 (blob ownership transfer across service domains) and H08 (blob cross-client sharing governance) are two instances of this pattern in the secure blob storage domain. H08 was driven by a concrete usage scenario in which a customer-facing channel receives documents that are later routed to support tooling; the support team needs read access without breaking the blob vault's ownership invariants. Both hypotheses were validated through a Guideline technical plan — an alignment session that produced a governance decision enabling prioritisation of the transfer and sharing implementation work. Quality attributes Security, Interoperability, and Usability are diagnostic markers for this pattern.

5.6 Revisiting the Research Questions

This section concludes the analytical body of the chapter by stating explicitly how the preceding empirical and design material bears on the four research questions formulated in Chapter 1. The intent is *consolidative*: no additional empirical claims are introduced; the aim is to make the logical organization of the chapter legible relative to those questions.

5.6.1 Correspondence between research questions and chapter sections

Table 5.5 summarises where each research question is principally addressed in this chapter, and where integrated interpretation (Group A/B) extends the principal sections.

RQ	Principal analysis	Integrated interpretation
RQ1	Section 5.1 (tool use and adoption dynamics).	Section 5.5.1 (Group A; read together with RQ3).
RQ2	Section 5.2 (typological account of the data).	Section 5.5.2 (Group B structural archetypes).
RQ3	Section 5.3 (benefits and challenges).	Section 5.5.1 (Group A; read together with RQ1).
RQ4	Section 5.4 (nine refinement categories, observation-led).	—

Table 5.5: *Correspondence between the research questions (Chapter 1) and the organization of evidence in this chapter.*

5.6.2 Brief interpretive synthesis

For RQ2, the question couples a typological obligation with a structural one. Section 5.2 discharges the former by characterising what was registered; Section 5.5.2 discharges the latter by articulating recurrent structural forms in the data. Jointly, they satisfy the formulation of RQ2 adopted in Chapter 1.

For RQ1 and RQ3, the Group A patterns provide an integrated reading: they condense conditions for sustaining hypothesis engineering that are visible only when observational evidence on use (RQ1) and on perceived benefits and friction (RQ3) is examined together. The wording of RQ1 and RQ3 is not revised here; the contribution is to clarify what an adequate empirical answer must cover in the present setting.

For RQ4, Section 5.4 reports refinements in the manner expected in design-science accounts of artefact evolution, that is, as outcomes grounded in observations rather than as a separate implementation chronicle.

Chapter 6

Conclusion

6.1 Summary in Relation to the Research Questions

RQ1.

RQ2. The study produced thirteen architectural hypotheses across two teams. Analysis of their source types, quality attributes, uncertainty and impact profiles, and technical plan strategies characterises the landscape of architectural uncertainty that Hypo-Stage users made explicit (Section 5.2). Beyond this characterisation, five recurring structural archetypes — the Group B patterns — were identified from the data: *Hypothesis Cluster*, *Regulatory Trigger Hypothesis*, *Decoupling Seam Hypothesis*, *Load-Validated Hypothesis*, and *Shared Resource Governance Hypothesis* (Section 5.5.2). These archetypes constitute the first empirically grounded answer to the question of what shapes architectural hypotheses take when ArchHypo is applied in a real industrial context, addressing a gap identified as future work in Silva *et al.* (2024).

RQ3.

RQ4.

6.2 Implications for Practice and Research

6.3 Limitations and Future Work

Appendix A

Research Instruments

A.1 Questionnaire

A.2 Interview Guide

A.3 Informed Consent Form

A.4 Training Communication

The following message was sent to teams before the training session that introduced the ArchHypo experiment and the Hypo-Stage plugin in Backstage.

> **New Plugin Experiment in Backstage: Hypo Stage — Practicing the ArchHypo Framework**
>
> Deciding when to make (or delay) a software architecture decision is one of the toughest daily challenges in engineering, regardless of the project:
>
> - Deciding too early can lock the system into the wrong path.
> - Deciding too late can lead to costly rework and technical debt.
>
> In the middle of this lies something that often remains implicit: uncertainty. We deal with requirements that are not yet clear or solutions we are not sure will scale or integrate well. This uncertainty is rarely made explicit, and decisions often end up being built on unvalidated premises.
>
> To evolve past this dilemma, I am proposing the application of a framework to help us identify, evaluate, and manage architectural uncertainties in a structured way. We will put this into practice using the Hypo Stage plugin, which I helped develop as part of an open-source project. It is already available in our Backstage (in production and under a feature flag for our experiment).
>
> This plugin serves as a tool to practice the ArchHypo (Architectural Hy-

potheses) framework: a lightweight approach that applies hypothesis-driven engineering to represent uncertainties impacting software architecture. It helps us plan actions to reduce uncertainty or impact before "locking in" a decision.

What ArchHypo proposes, in short:

Architectural Hypotheses: Falsifiable statements expressing team beliefs about requirements (e.g., quality attributes) or current/candidate solutions. The focus is on uncertainties that affect architectural decisions, not just business hypotheses.

Evaluation: Each hypothesis is assessed based on its uncertainty level (how far we are from proving it true/false) and impact (the consequences and cost of being wrong). This helps prioritize and choose treatment tactics.

Technical Plan: For each hypothesis, the team defines actions to either reduce uncertainty (e.g., spikes, experiments, architectural triggers, metrics) or reduce impact (e.g., modularity, flexibility, isolation of the affected part), allowing us to delay decisions more safely.

Iterative Cycle: Identify, evaluate, plan, execute, and reassess. The process is continuous; as new uncertainties emerge, plans are adjusted over time.

The Hypo Stage plugin in Backstage allows you to register hypotheses, link them to quality attributes, record uncertainty/impact assessments, and maintain technical plans with dates and progress tracking — all integrated into our catalog and workflow.

Want to better understand the problem, the framework, and how to use the plugin?

This Week Day, Calendar Date, at HH:MM during the Meeting Agenda, I will be leading a session on this topic. We will discuss why finding the right moment for architectural decisions is so difficult and how ArchHypo can help.

I am looking forward to seeing you there to discuss this experiment and how to use Hypo Stage in our daily routine.

Appendix B

Registered Hypotheses Data

This appendix presents the complete set of thirteen architectural hypotheses registered by the two participating teams in Hypo-Stage during the observation period. All entity references, service names, and internal artefact links have been anonymised in accordance with the ethical agreement described in Section 4.9. The anonymisation legend is provided in Table B.1. Technical planning entries are reproduced in full in Section B.3.

B.1 Anonymisation Legend

Hypotheses in Hypo-Stage are linked to components in the organization's Backstage catalog through `entityRef` strings (human-readable labels that identify a service, worker, platform capability, or template entry). To protect confidentiality under the ethical agreement summarised in Section 4.9, this appendix does not reproduce real catalog identifiers. Instead, each hypothesis table in Section B.2 lists *anonymised* entity references in its **Entities** row; Table B.1 explains what each anonymised string denotes at a functional level, without recovering the original catalog paths or service names.

The table is intentionally typeset here as *non-floating* material so it remains with this section even though it occupies most of a page: the goal is to keep the legend immediately available when reading the hypothesis registers, rather than deferring it past the large tables in Section B.2.

Anonymised entity reference	Description
`catalog: Kotlin + JOOQ + Spring service template`	Internal catalog entry for the JVM service scaffold using Kotlin, JOOQ, and Spring — intended to replace a legacy functional-language stack in the regulated financial domain.
`internal-framework: JVM platform stack with Kafka integration`	The organization's shared application framework for JVM services, covering Kafka exposure conventions, request logging, and platform integrations.
`service: secure blob storage — API`	HTTP API for storing and governing binary documents with security and lifecycle rules.
`service: secure blob storage — worker`	Background worker for the secure blob storage domain (async processing and maintenance).
`service: confidential records & credit-bureau integration — API`	API for identity, credit-bureau, and confidential document flows, currently coupled to a single business vertical.
`service: confidential records & credit-bureau integration — worker`	Worker for the same confidential records and bureau integration context.
`worker: high-volume billing aggregation`	Long-running worker that aggregates large billing datasets.
`platform: shared streaming infrastructure`	Shared Kafka streaming platform and consumption standards, modelled as a catalog component.

Table B.1: *Anonymised Backstage entity references and their descriptions.*

B.2 Hypothesis Registers

Each hypothesis is typeset as a single register table: fixed metadata rows, a full-width statement row, then author notes and technical-plan references (see Section B.3). The bold header on each register repeats the hypothesis identifier and short label exactly as in Table 5.1 of Chapter 5.

H01 — Kotlin/JOOQ/Spring template readiness

Team	Product
Registered	2026-03-18
Lifecycle status	Open
Source type	Requirements
Uncertainty / Impact	Very High / Very High
Quality attributes	Performance | Reliability | Maintainability
Entities	`catalog: Kotlin + JOOQ + Spring service template`

Statement

"The Kotlin + JOOQ + Spring service template is ready to scale for use in regulated financial product services, replacing a legacy functional-language stack."

Notes	The organization intends to deprecate the legacy stack in that domain; the Kotlin + JOOQ + Spring template is the leading candidate to replace it.
Technical plans	None registered at end of observation period.

H02 — Kafka direct exposure via platform framework

Team	Platform
Registered	2026-02-11
Lifecycle status	Open
Source type	Requirements
Uncertainty / Impact	Very Low / Medium
Quality attributes	Configurability | Flexibility | Extensibility | Modifiability
Entities	`internal-framework: JVM platform stack with Kafka integration`

Statement

"Kafka should be exposed to application layers for direct use via the internal platform framework."

Notes	Subject on hold; technical plans expected for the following quarter.
Technical plans	None registered at end of observation period.

H03 — Decouple eng. logs from security/compliance audit

Team	Platform
Registered	2026-02-19
Lifecycle status	Open
Source type	Requirements
Uncertainty / Impact	Medium / Very High
Quality attributes	Observability | Auditability
Entities	`internal-framework: JVM platform stack with Kafka integration`

Statement

"Decouple engineering/observability logs from security/compliance auditing by evolving database logging toward a unified Request Logs capability on the internal platform framework."

Notes	The platform framework must let applications derive full log context including database logs. It is expected to supersede a legacy audit library and satisfy organization-wide security audit requirements.
Technical plans	TP-03-A, TP-03-B, TP-03-C (see Section B.3).

H04 — Blob storage GDPR right-to-erasure

Team	Product
Registered	2026-03-18
Lifecycle status	Open
Source type	Requirements
Uncertainty / Impact	Medium / Very Low
Quality attributes	Auditability | Usability
Entities	`service: secure blob storage — API | service: secure blob storage — worker`

Statement

"The secure blob storage service must retain a reference to the data subject in its domain data so all sensitive blobs can be deleted when the user exercises their right to erasure under applicable privacy regulation."

Notes	Concern raised in an internal RFC about fully erasing user sensitive data, including binary objects.
Technical plans	None registered at end of observation period.

H05 — Credit-bureau decoupling seam

Team	Product
Registered	2026-03-18
Lifecycle status	In Review
Source type	Solution
Uncertainty / Impact	High / Very High
Quality attributes	Interoperability | Extensibility | Reusability
Entities	`service: confidential records & credit-bureau integration — API | ... worker`

Statement

"Credit-bureau integrations can be decoupled from vertical-specific domain entities into context-agnostic middleware that accepts only each provider's minimum required data (e.g., national ID), reusable across domains without duplicated cost."

Notes	Discovery focused on: (1) dependency matrix — current vs. real dependency per integration; (2) agnostic interface — contract without domain concepts, only minimal identifiers and bureau outputs; (3) migration plan — which flows can isolate immediately vs. requiring deeper business-logic refactors.
Technical plans	TP-05-A (see Section B.3).

H06 — Billing aggregation chunked keyset pagination

Team	Product
Registered	2026-02-24
Lifecycle status	Open
Source type	Solution
Uncertainty / Impact	Very Low / Very High
Quality attributes	Performance | Reliability | Scalability
Entities	`worker: high-volume billing aggregation`

Statement

"Chunked keyset pagination with isolated repository-level transactions keeps memory strictly bounded by chunk size and prevents out-of-memory failures when aggregating millions of billing records."

Notes	Validated with a capacity/volume test at 5× production scale (45M rows total, 1.5M rows/month). Profiling showed JPA streams can cause first-level cache growth; chunking scopes transactions (≈10k rows) so the garbage collector can reclaim memory (saw-tooth heap pattern). Composite index on `(billing_year_month, status, id)`.
Technical plans	TP-06-A (see Section B.3).

H07 — Blob ownership transfer across service domains

Team	Product
Registered	2026-04-07
Lifecycle status	Validated
Source type	Requirements
Uncertainty / Impact	Very Low / Very High
Quality attributes	Scalability | Usability | Configurability | Flexibility
Entities	`service: secure blob storage – API` | `service: secure blob storage – worker`

Statement

"Secure blob storage must support transferring blob ownership across service domains so blobs can move from one owning service to another."

Notes	Validated through a technical discovery spike that defined the transfer capability and produced a prioritised implementation backlog.
Technical plans	TP-07-A (see Section B.3).

H08 — Blob cross-client sharing governance

Team	Product
Registered	2026-03-18
Lifecycle status	Validated
Source type	Requirements
Uncertainty / Impact	Very Low / Very High
Quality attributes	Security | Usability | Interoperability | Extensibility
Entities	`service: secure blob storage – API`

Statement

"Secure blob storage must support sharing a blob across multiple clients so the same object can be read from different domain services."

Notes	Example: a customer-facing channel receives documents later routed to support tooling; support needs read access in chat history without breaking vault invariants.
Technical plans	TP-08-A (see Section B.3).

H09 — DLT for audit vs. offset-based replay

Team	Platform
Registered	2026-04-12
Lifecycle status	Open
Source type	Solution
Uncertainty / Impact	Medium / Very High
Quality attributes	Auditability | Reliability
Entities	`platform: shared streaming infrastructure`

Statement

"Using the Dead-Letter Topic (DLT) only for auditing and diagnosis, and reprocessing from the original topic's offset, preserves consistency and auditability better than re-enqueueing messages from the DLT."

Notes	How to falsify: controlled replay from the DLT without mutating the event must be proven safer, simpler, and more auditable than replay by offset.
Technical plans	TP-09-A, TP-09-B (see Section B.3).

H10 — Synchronous retry for transient consumer failures

Team	Platform
Registered	2026-04-12
Lifecycle status	Open
Source type	Solution
Uncertainty / Impact	Medium / High
Quality attributes	Interoperability | Performance
Entities	`platform: shared streaming infrastructure`

Statement

"For short-lived transient failures, limited synchronous retry on the consumer thread preserves per-partition ordering better and with lower overall complexity than asynchronous retry on a separate queue."

Notes	How to falsify: a per-partition retry queue preserves order with less blocking time and acceptable operational cost.
Technical plans	TP-10-A (see Section B.3).

H11 — Failure state inside vs. outside Kafka flow

Team	Platform
Registered	2026-04-12
Lifecycle status	Open
Source type	Solution
Uncertainty / Impact	Medium / High
Quality attributes	Reliability | Modifiability | Interoperability
Entities	`platform: shared streaming infrastructure`

Statement

"Keeping failure state inside the Kafka message flow increases coordination complexity across threads and complicates failure handling, compared to handling failures in the execution layer and keeping consumption non-blocking."

Notes	How to falsify: modelling failure state in Kafka shows lower operational complexity and equal or better recovery than handling failures in the execution layer.
Technical plans	TP-11-A (see Section B.3).

H12 — Error classification before retry

Team	Platform
Registered	2026-04-12
Lifecycle status	Open
Source type	Solution
Uncertainty / Impact	Medium / High
Quality attributes	Performance | Reliability
Entities	`platform: shared streaming infrastructure`

Statement

"Classifying errors as recoverable vs. non-recoverable before applying retry reduces consumer lag and computational cost without worsening the successful processing rate."

Notes	How to falsify: indiscriminate retries improve final success rate without meaningful latency/capacity cost.
Technical plans	TP-12-A, TP-12-B (see Section B.3).

H13 — Payload immutability before re-enqueueing

Team	Platform
Registered	2026-04-12
Lifecycle status	Open
Source type	Solution
Uncertainty / Impact	Very Low / Very High
Quality attributes	Auditability
Entities	`platform: shared streaming infrastructure`

Statement

"Allowing fixes by mutating the payload before re-enqueueing increases consistency violations and makes traceability and auditing harder."

Notes	How to falsify: a formal correction mechanism fully preserves audit trail, causality, and reproducibility of the event.
Technical plans	TP-13-A (see Section B.3).

B.3 Technical Plans Catalog

The following entries reproduce all technical plans registered in Hypo-Stage during the observation period. Each entry is identified by a plan code (e.g., TP-03-A), the parent hypothesis, the action type, the target date, a description of the planned action, and the expected outcome. Internal artefact links (tickets, pull requests, design documents) have been redacted.

TP-03-A (H03 — Decouple eng. logs from security/compliance audit) *Action type:* Architectural Spike *Target date:* 2026-03-31
Description: Discover and implement default mapping of database transaction IDs into the Request Log structure within the internal framework so every request and application log entry carries database transaction IDs for correlation and audit trail without per-application integration.
Expected outcome: Request logs consistently include database transaction ID fields; applications using the framework obtain audit-ready context without custom wiring.

TP-03-B (H03) *Action type:* Modularisation / Flexibility *Target date:* 2026-03-31
Description: Extend framework logging to worker modules so critical database transactions in workers log with the same context (transaction IDs, correlation) as request paths.
Expected outcome: Workers emit logs with database transaction IDs aligned with the Request Log infrastructure; audit coverage for asynchronous/background work.

TP-03-C (H03) *Action type:* Modularisation / Flexibility *Target date:* 2026-03-31
Description: Stream database transaction properties to the same log buckets and pipeline as Request and Application logs for a single audit/observability backbone.

Expected outcome: Database transaction properties follow the same ingestion and retention path as Request logs; no separate audit-only pipeline.

TP-05-A (H05 — Credit-bureau decoupling seam) *Action type:* Discovery *Target date:* ongoing
Description: Structured discovery covering: (1) dependency matrix (current vs. real dependencies per integration); (2) agnostic interface proposal (contract free of domain concepts, accepting only minimal provider identifiers and bureau outputs); (3) migration sequencing (which flows can isolate immediately vs. requiring deeper business-logic refactors).
Expected outcome: A prioritised integration decoupling plan and a draft interface contract.

TP-06-A (H06 — Billing aggregation chunked keyset pagination) *Action type:* Implementation (Other) *Target date:* 2026-03-13
Description: Implement the billing aggregation worker with chunked keyset pagination using a `LIMIT` loop with `id > :lastProcessedId`; avoid broad `@Transactional` so the Hibernate first-level cache clears per chunk; add composite index on `(billing_year_month, status, id)`.
Expected outcome: Process 300k+ rows/month safely without out-of-memory failures; stable sawtooth heap profile; healthy database CPU/I/O via index scans.

TP-07-A (H07 — Blob ownership transfer across service domains) *Action type:* Architectural Spike *Target date:* 2026-04-30
Description: Technical discovery to define the transfer capability on the blob storage API.
Expected outcome: A backlog for implementing the ownership transfer feature.

TP-08-A (H08 — Blob cross-client sharing governance) *Action type:* Guideline Implementation *Target date:* 2026-04-06
Description: Alignment session on cross-domain sharing and transfers so that long-lived access (e.g., messaging history) does not require definitive ownership by every consumer.
Expected outcome: A governance decision enabling prioritisation of sharing and transfer implementation work.

TP-09-A (H09 — DLT for audit vs. offset-based replay) *Action type:* Guideline Implementation *Target date:* 2026-06-30
Description: Produce a runbook for reprocessing by offset with minimal tooling for controlled replay.
Expected outcome: Documentation of how to reprocess by offset with minimal tooling.

TP-09-B (H09) *Action type:* Experiment *Target date:* 2026-06-30
Description: Operational exercise with a simulated incident.
Expected outcome: Measure MTTR, duplicate rate, and divergence between the original event and the final state.

TP-10-A (H10 — Synchronous retry for transient failures) *Action type:* Tracer Bullet *Target date:* 2026-06-30

Description: Tracer bullet on a critical consumer path; inject transient failures with windows of 1s, 5s, and 30s.
Expected outcome: Measure out-of-order delivery, consumer lag, throughput, and retry success rate.

TP-11-A (H11 — Failure state inside vs. outside Kafka flow) *Action type:* Architectural Spike *Target date:* 2026-06-30
Description: Spike comparing both failure-state approaches on a real consumer.
Expected outcome: Measure consumer lag, blocked-thread time, states/transitions, and operational recovery steps for each approach.

TP-12-A (H12 — Error classification before retry) *Action type:* Guideline Implementation *Target date:* 2026-06-30
Description: Define an error taxonomy and mandatory classification step before retry is applied.
Expected outcome: Practical directives for classifying errors before retry, consumable by all teams using the shared streaming platform.

TP-12-B (H12) *Action type:* Software Analytics (Architectural Trigger) *Target date:* 2026-06-30
Description: Instrument analytics by error class on a real consumer: success rate, average attempts, extra time per message, and discards to DLT.
Expected outcome: A metrics dataset sufficient to falsify or support the hypothesis.

TP-13-A (H13 — Payload immutability before re-enqueueing) *Action type:* Experiment *Target date:* 2026-06-30
Description: Enforce an immutable-payload guideline; add automated validation in replay tooling; require mandatory architectural review for exceptions.
Expected outcome: Experiment results that validate or refute the hypothesis and establish a baseline for payload governance in the shared streaming platform.

References

[L. Bass *et al.* 2021] Len Bass, Paul Clements, and Rick Kazman. *Software Architecture in Practice*. 4th ed. Addison-Wesley, 2021 (cit. on p. 9).

[Betts *et al.* 2024] Thomas Betts, Blanca García Gil, Eran Stiller, Daniel Bryant, and Rafal Gancarz. *InfoQ Software Architecture and Design Trends Report — 2024*. https://www.infoq.com/articles/architecture-trends-2024/. Grey literature — practitioner report. Accessed: 23 March 2026. 2024 (cit. on pp. 1, 2, 4, 6, 22, 23).

[Borowa *et al.* 2023] Klara Borowa, Rafał Lewanczyk, Klaudia Stpiczynska, Patryk Stradomski, and Andrzej Zalewski. "What rationales drive architectural decisions? an empirical inquiry". In: *Proceedings of the 17th European Conference on Software Architecture (ECSA)*. Springer, 2023. url: https://arxiv.org/pdf/2309.14164.pdf (cit. on p. 22).

[Conway 1968] Melvin E. Conway. "How do committees invent?" *Datamation* 14.4 (1968), pp. 28–31 (cit. on p. 22).

[Corrêa 2025] Pedro Henrique Mariano Corrêa. *ArchHypo Tooling: Enabling Practical Hypothesis Engineering for Software Architecture*. Capstone Project Report (Bachelor), Institute of Mathematics and Statistics, University of São Paulo. Supervised by José G. Lima Neto; co-supervised by Paulo Meirelles. 2025 (cit. on pp. viii, 3–7, 16–19, 43).

[Da Silveira Neto *et al.* 2025] Manoel Valério Da Silveira Neto, Andreia Malucelli, and Sheila Reinehr. "Agile software architecture: perceptions on quality and architectural technical debt management". In: *Proceedings of the XIX Brazilian Symposium on Software Components, Architectures, and Reuse (SBCARS)*. SBC, 2025. doi: 10.5753/sbcars.2025.4769 (cit. on p. 21).

[Dasanayake *et al.* 2017] Sandun Dasanayake, Jouni Markkula, Sanja Aaramaa, and Markku Oivo. "An empirical study on collaborative architecture decision making in software teams". In: *Proceedings of the 11th European Conference on Software Architecture (ECSA) Companion*. Springer, 2017. url: https://arxiv.org/pdf/1707.00107.pdf (cit. on p. 21).

[Foote and Yoder 1999] Brian Foote and Joseph Yoder. "Big ball of mud". *Pattern Languages of Program Design* 4 (1999), pp. 654–692 (cit. on p. 10).

[Herbsleb and Grinter 1999] James D. Herbsleb and Rebecca E. Grinter. "Splitting the organization and integrating the code: Conway's law revisited". In: *Proceedings of the 21st International Conference on Software Engineering (ICSE)*. ACM/IEEE, 1999, pp. 85–95. URL: https://www.cs.cmu.edu/afs/cs.cmu.edu/Web/People/jdh/collaboratory/research_papers/ICSE99.pdf (cit. on p. 22).

[Hunter and Stojmilova 2025] Peter Hunter and Elena Stojmilova. "Empowering teams: decentralizing architectural decision-making". *InfoQ* (Nov. 2025). Grey literature — practitioner article. URL: https://www.infoq.com/articles/empowering-decentralizing-architectural-decision-making/ (cit. on p. 23).

[Oliveira De Diana *et al.* 2011] Mauricio José de Oliveira De Diana, Fabio Kon, and Marco Aurélio Gerosa. "Conducting an architecture group in a multi-team agile environment". In: *Proceedings of the 5th Brazilian Symposium on Software Components, Architectures and Reuse (SBCARS)*. SBC, 2011. URL: https://www.ime.usp.br/~kon/papers/arch_group.pdf (cit. on p. 21).

[Perry and Wolf 1992] Dewayne E. Perry and Alexander L. Wolf. "Foundations for the study of software architecture". *ACM SIGSOFT Software Engineering Notes* 17.4 (1992), pp. 40–52. DOI: 10.1145/141874.141884 (cit. on p. 1).

[Razavian 2019] Maryam Razavian. "Empirical research for software architecture decision making: an analysis". *Journal of Systems and Software* 149 (2019), pp. 360–381. DOI: 10.1016/j.jss.2018.12.013 (cit. on p. 22).

[Richards and Ford 2020] Mark Richards and Neal Ford. *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media, 2020. ISBN: 9781492043454 (cit. on p. 9).

[Robson and McCartan 2016] Colin Robson and Kieran McCartan. *Real World Research: A Resource for Users of Social Research Methods in Applied Settings*. 4th. John Wiley & Sons, 2016. ISBN: 978-1-118-74523-6 (cit. on pp. ix, 27, 28, 30, 33–36).

[Salameh and J. M. Bass 2021] Abdallah Salameh and Julian M. Bass. "An architecture governance approach for Agile development by tailoring the Spotify model". *AI & Society* 37 (2021), pp. 1225–1243. DOI: 10.1007/s00146-021-01240-x (cit. on p. 21).

[Silva *et al.* 2024] Eduardo Silva *et al.* "Patterns for using hypothesis engineering to manage architectural uncertainty". In: *Proceedings of the Latin American Conference on Pattern Languages of Programming (SugarLoafPLoP)*. 2024. DOI: 10.1145/3698322.3698333 (cit. on pp. 6, 40, 46, 48, 53).

[Silva, Melegati, Silveira, *et al.* 2025] Eduardo Silva, Jorge Melegati, Moises Silveira, *et al.* "ArchHypo: managing software architecture uncertainty using hypotheses engineering". *IEEE Transactions on Software Engineering* 51.2 (2025), pp. 481–499. DOI: 10.1109/TSE.2024.3520477 (cit. on pp. viii, 1–4, 10–15, 24, 28, 32, 38, 41, 42, 44, 48).

[Silva, Melegati, Wang, *et al.* 2024] Eduardo Silva, Jorge Melegati, Xiaofeng Wang, and Paulo Meirelles. "Using hypotheses to manage technical uncertainty and architecture evolution in a software start-up". *IEEE Software* 41.4 (2024), pp. 78–87. doi: 10.1109/MS.2024.3383628 (cit. on pp. viii, 1–4, 10, 12, 13, 15, 24, 38, 42, 48).

[Skelton and Pais 2019] Matthew Skelton and Manuel Pais. *Team Topologies: Organizing Business and Technology Teams for Fast Flow*. IT Revolution Press, 2019 (cit. on pp. 9, 10, 16, 22, 23).

[Souza *et al.* 2019] Eric Souza, Ana Moreira, and Miguel Goulão. "Deriving architectural models from requirements specifications: a systematic mapping study". *Information and Software Technology* 109 (2019), pp. 26–39. doi: 10.1016/j.infsof.2019.01.004 (cit. on pp. 1–4, 6, 7, 9, 19, 20, 22).

[Spotify and Cloud Native Computing Foundation 2024] Spotify and Cloud Native Computing Foundation. *Backstage: An Open Source Framework for Building Developer Portals*. https://backstage.io. Open-source project; over 3,000 companies adopted as of 2025. CNCF Incubating Project. 2024 (cit. on p. 16).

[Tang *et al.* 2017] Antony Tang, Maryam Razavian, Barbara Paech, and Tom-Michael Hesse. "Human aspects in software architecture decision making: a literature review". In: *2017 IEEE International Conference on Software Architecture (ICSA)*. IEEE, 2017, pp. 107–116. doi: 10.1109/ICSA.2017.27 (cit. on p. 22).

[Tibbitts 2025] Lindsey Tibbitts. "Decentralized architecture needs more than autonomy". *InfoQ* (June 2025). Grey literature — practitioner article. url: https://www.infoq.com/articles/decentralized-architecture-advice-process/ (cit. on p. 23).

[Yin 2009] Robert K. Yin. *Case Study Research: Design and Methods*. 4th. Thousand Oaks, CA: SAGE Publications, 2009 (cit. on pp. 27, 28, 30, 33, 35).